

Architectures and Platforms for AI module 1

Pacheco, Chapter 1

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

<https://www.moreno.marzolla.name/>

Copyright © 2013—2020, 2024, 2025
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/APAI/>



This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- prof. Salvatore Orlando, Univ. Ca' Foscari di Venezia
<http://www.dsi.unive.it/~orlando/>
- prof. Mary Hall, University of Utah
<https://www.cs.utah.edu/~mhall/>
- Tim Mattson, Intel

Who I am

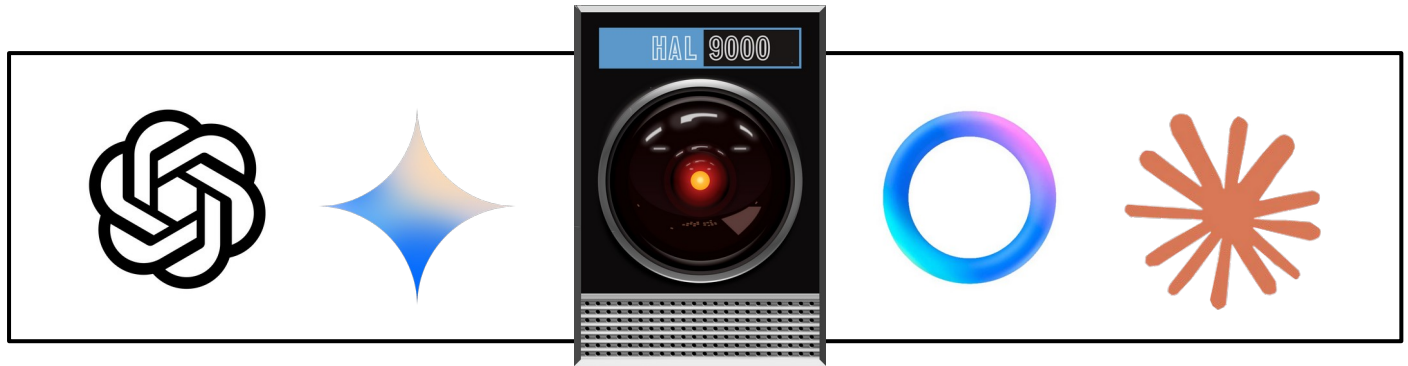
- Moreno Marzolla
 - moreno.marzolla@unibo.it
 - <https://www.moreno.marzolla.name/>
- Teaching
 - High Performance Computing
 - Architectures and Platforms for Artificial Intelligence
- Research activity
 - Parallel programming
 - Distributed systems
 - Modeling and simulation

Architectures and Platforms for AI module 1

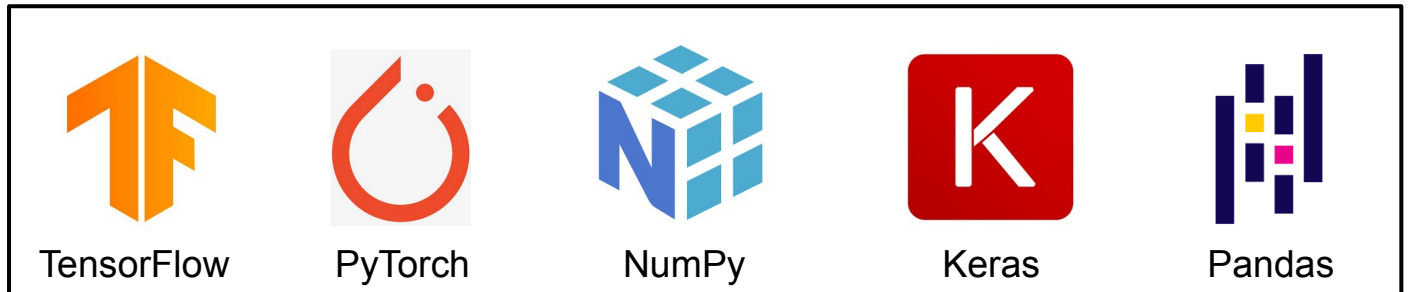
- Web page
 - <https://www.moreno.marzolla.name/teaching/APAI>
- Schedule
 - Monday 09:30—12:30 Room III
 - Please check for last-minute changes
 - the Web page
 - the official course timetable for last-minute changes
 - the “General course announcements” on *Virtuale*
- Office hours
 - At the end of lectures
 - On demand via Teams; please [send e-mail](#)

Is this course right for you?

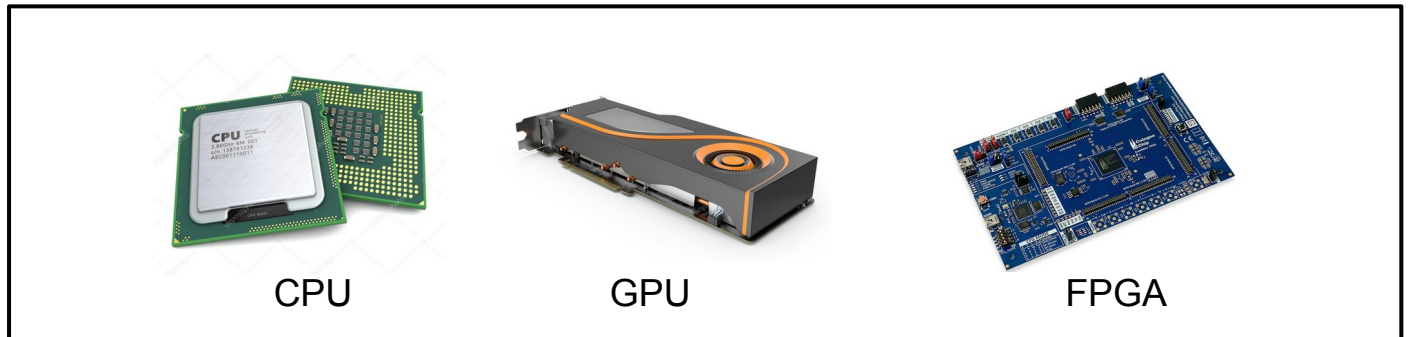
Applications



Python
Libraries



Hardware

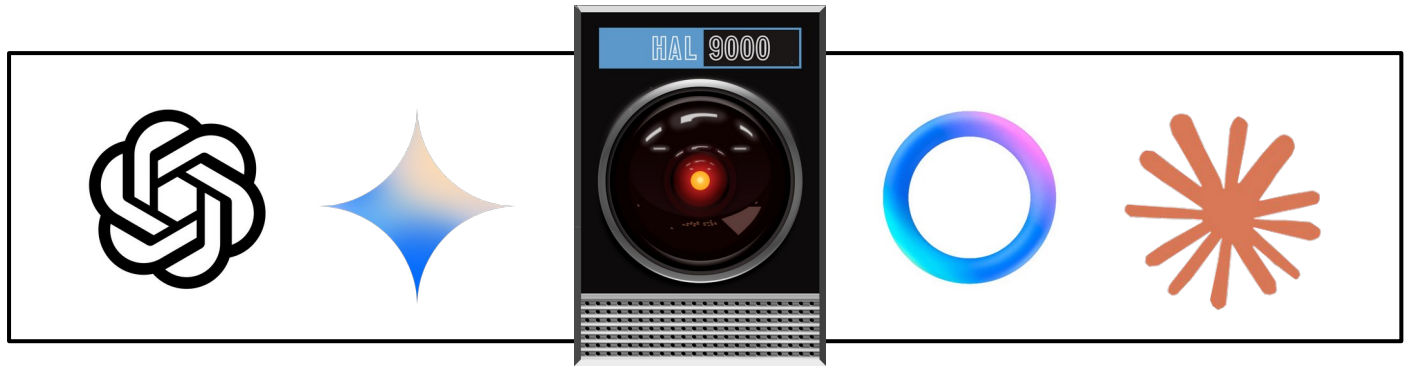


Architectures and Platforms for AI

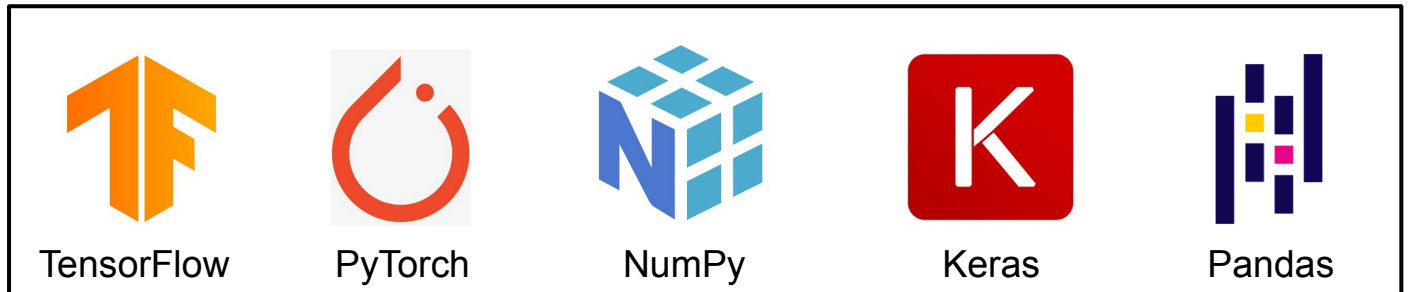
Logos are © their owners.

Is this course right for you?

Applications



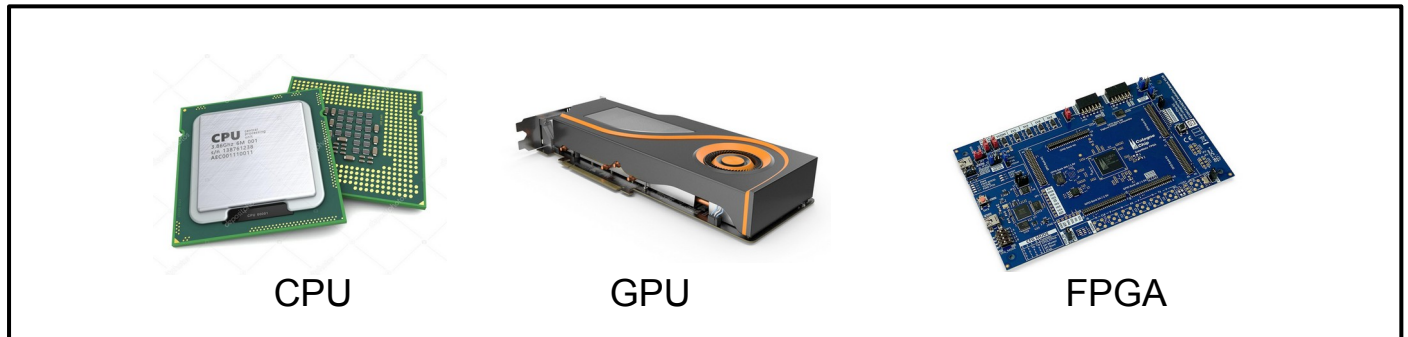
Python
Libraries



Low-level
C code

BLAS, K-Means, Scan/Reduce, ...

Hardware



Architectures and Platforms for AI

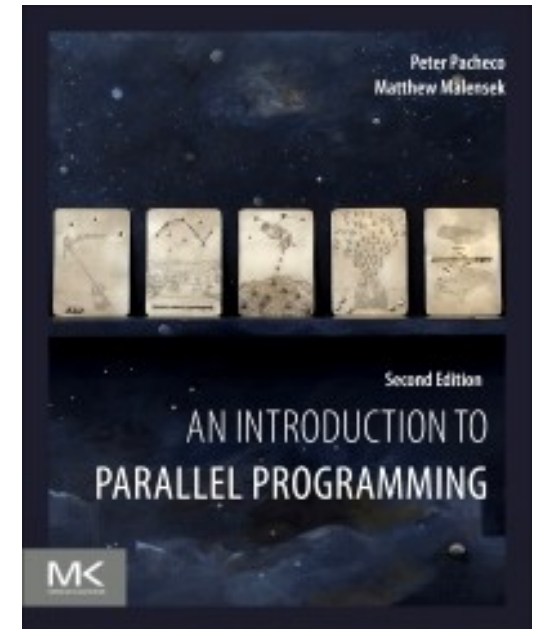
Logos are © their owners.

Syllabus for Module 1

- Theory
 - Parallel programming fundamentals
 - Parallel programming patterns
 - Performance evaluation of parallel programs
- Parallel programming practice
 - Shared-memory programming with OpenMP
 - GPU programming with CUDA

Textbook

- Peter Pacheco, Matthew Malensek,
An Introduction to Parallel Programming, 2nd ed., Morgan Kaufmann, 2021, ISBN 9780128046050
 - The 1st edition is ok, too
- *CUDA C programming guide*
<http://docs.nvidia.com/cuda/cuda-c-programming-guide/>
 - CUDA/C programming
- Slides and other material
<https://www.moreno.marzolla.name/teaching/APAI/>



Prerequisites

- Mandatory
 - C programming
 - Required for the final project; no Python, Java, ... code will be accepted
- Optional, but highly recommended
 - Use of the Unix/Linux environment
 - Basic notions of
 - Computer Architectures
 - Algorithms and Data Structures
 - Concurrency theory

Exam (module 1)

- Independent from module 2
- Programming project + written report
 - To be done individually
 - Using OpenMP and/or CUDA (will depend on the level of difficulty)
 - Specifications defined by the instructor, discussed during the last lecture of the module; will remain valid for the whole academic year
 - The project can be handed in at any time until September 30, 2026
 - There is no oral discussion, unless I request one
- Final grade
 - Average of modules 1 and 2, rounded to nearest integer
 - Honors (“*lode*”) assigned by the instructors for outstanding work
- Partial grades remain valid until September 30, 2026
 - After that, a new edition of this course starts

Exam

- To register the final grade, you **must** pass the exam for **both** module 1 and 2
 - Partial credit can not be given

Grading the programming project

- Quality of the implementation
 - Correctness
 - Readability
 - Efficiency
- Quality of the written report
 - Proper grammar, syntax, ...
 - Technical correctness
 - Performance evaluation

Generative AI policy

- The instructor did not use generative AI during the preparation of the teaching material
- **Generative AI shall not be used in any part of the final project (code and report)**
 - Spell-checking and other generic writing assistants are allowed



Programming examples

- BYOD (Bring Your Own Device)
- You can get remote access (ssh) to a Linux-based HPC cluster managed by the Computer Science and Engineering Department
 - See the instructions on *Viruale* on how to get an account

Questions?

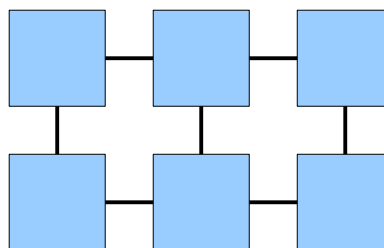
Intro to parallel programming

High Performance Computing

- Many applications need considerable computing power
 - Weather forecast, climate modeling, physics simulation, 3D animation, finance, AI training/inference...
- Why?
 - To solve more complex problems
 - To solve the same problem in less time
 - To solve the same problem more accurately
 - To make better use of available computing resources

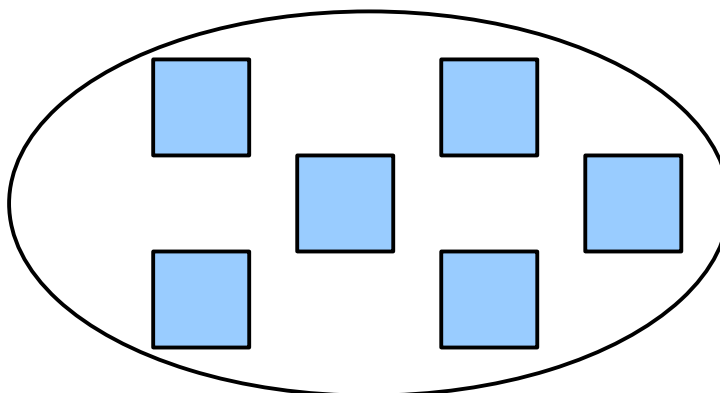
Parallel Computing (HPC) vs Cloud Computing (HTC)

- In **High Performance Computing (HPC)** we break applications into **strongly coupled** pieces executed on **specialized hardware** (CPUs and/or interconnection networks)



Interconnection network is important

- In **High Throughput Computing (HTC)** we break applications into **loosely coupled** pieces executed on **commodity hardware**



Interconnection network is less important

High Performance Computing

- “Traditional” scientific paradigm
 - Make a theory, then experiment
- “Traditional” engineering paradigm
 - Design, then build
- Enter numeric experimentation and prototyping
 - Some phenomena are too complex to be modeled accurately (e.g., weather forecast)
 - Some experiments are too complex, or costly, or dangerous, or impossible to do in the lab (e.g., wind tunnels, seismic simulations, stellar dynamics...)
- *Computational science*
 - Numerical simulations are becoming a new way to “do science”

Applications: Molecular dynamics

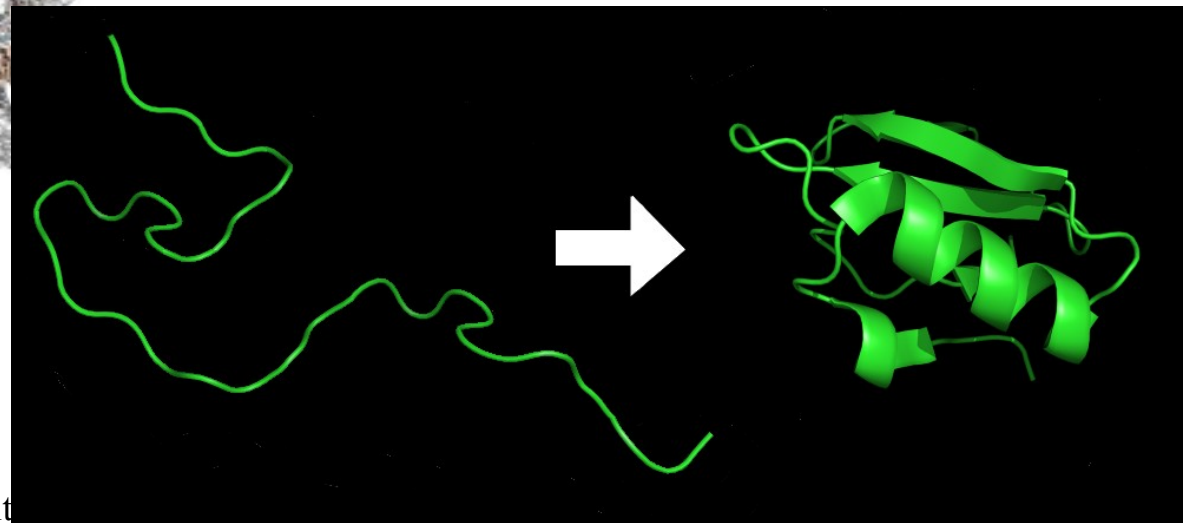
Article | Published: 24 May 2021

SARS-CoV-2 simulations go exascale to predict dramatic spike opening and cryptic pockets across the proteome

[Maxwell I. Zimmerman](#), [Justin R. Porter](#), [Michael D. Ward](#), [Sukrit Singh](#), [Neha Vithani](#), [Artur Meller](#), [Upasana L. Mallimadugula](#), [Catherine E. Kuhn](#), [Jonathan H. Borowsky](#), [Rafal P. Wiewiora](#), [Matthew F. D. Hurley](#), [Aoife M. Harbison](#), [Carl A. Fogarty](#), [Joseph E. Coffland](#), [Elisa Fadda](#), [Vincent A. Voelz](#), [John D. Chodera](#) & [Gregory R. Bowman](#) ✉

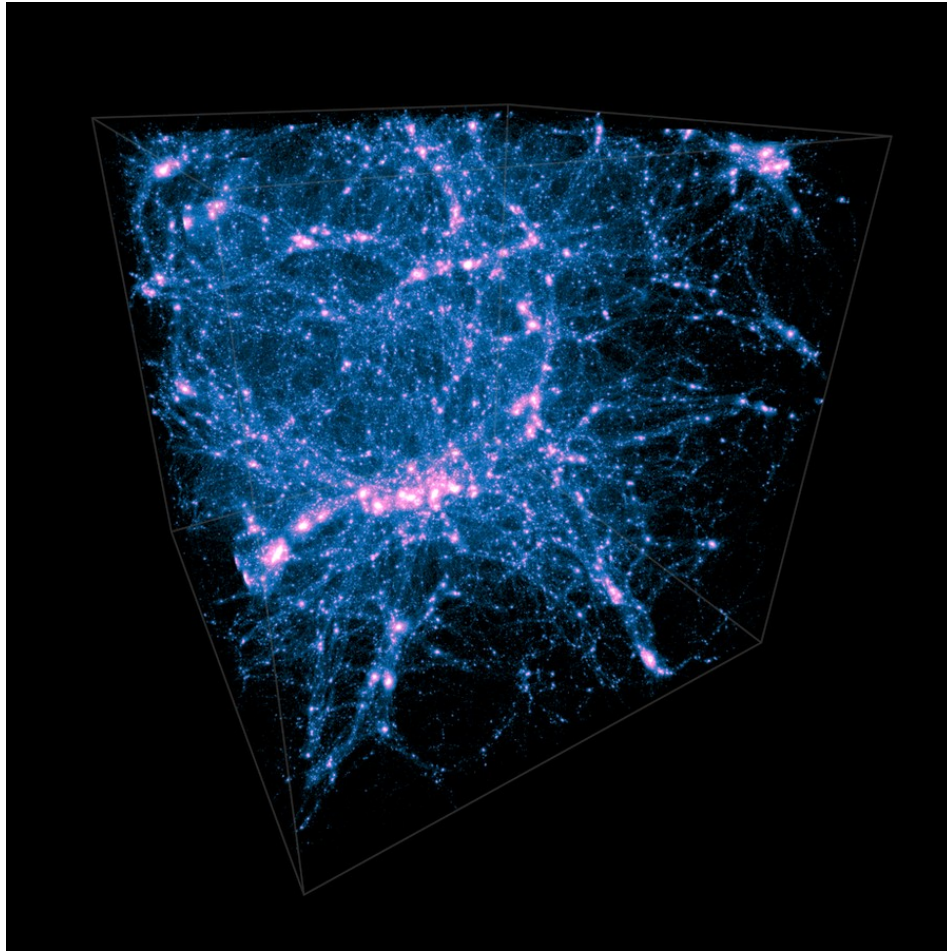
[Nature Chemistry](#) **13**, 651–659 (2021) | [Cite this article](#)

Source: <http://bionano.physics.illinois.edu/node/109>



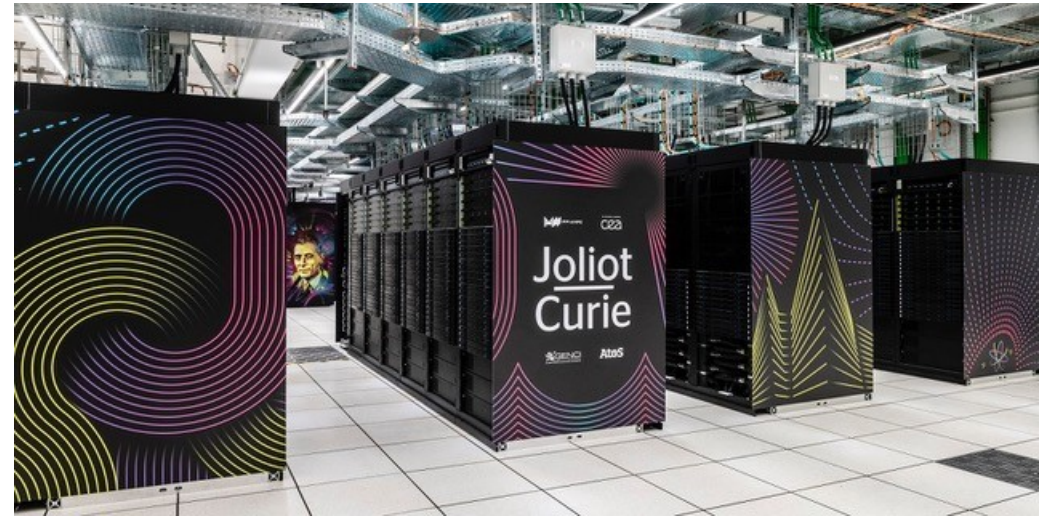
Archit

Cosmological simulation: the Illustris project



“The Illustris simulations were run on supercomputers in France, Germany, and the US. The largest was run on 8,192 compute cores, and took 19 million CPU hours”

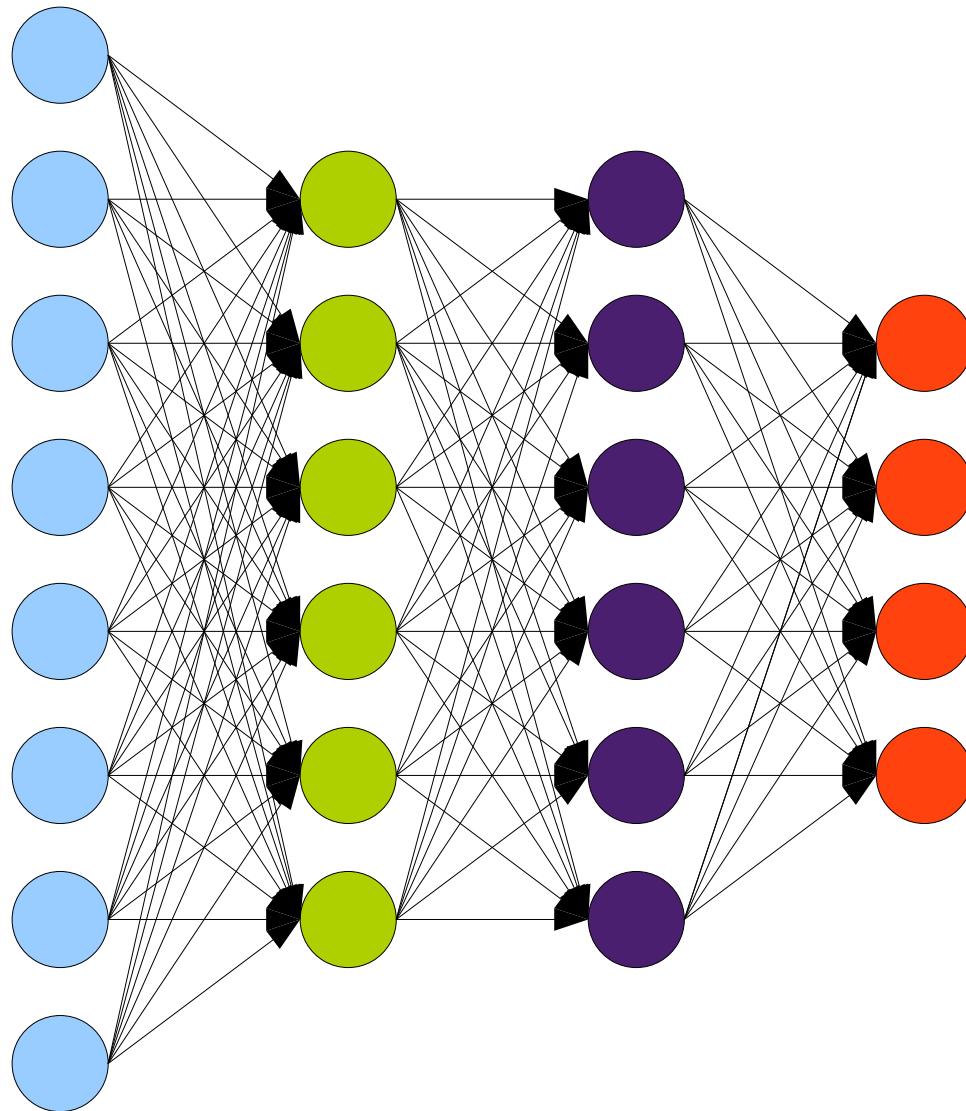
https://www.illustris-project.org/movies/illustris_movie_rot_sub_frame.mp4



Most of the Illustris simulation was executed in the Curie supercomputer with 440,000 x86 cores in nearly 5,000 nodes, 22 petaflop/s peak computing power

```
for each particle p {  
    compute the force on p by all other  
    particles p';  
    compute the new position of p using  
    Newton's second law;  
}
```

Applications: Neural Networks



Moore's Law

"The number of transistors on an IC doubles every 24 months"

- That **used** to mean that every new generation of processors was based on smaller transistors

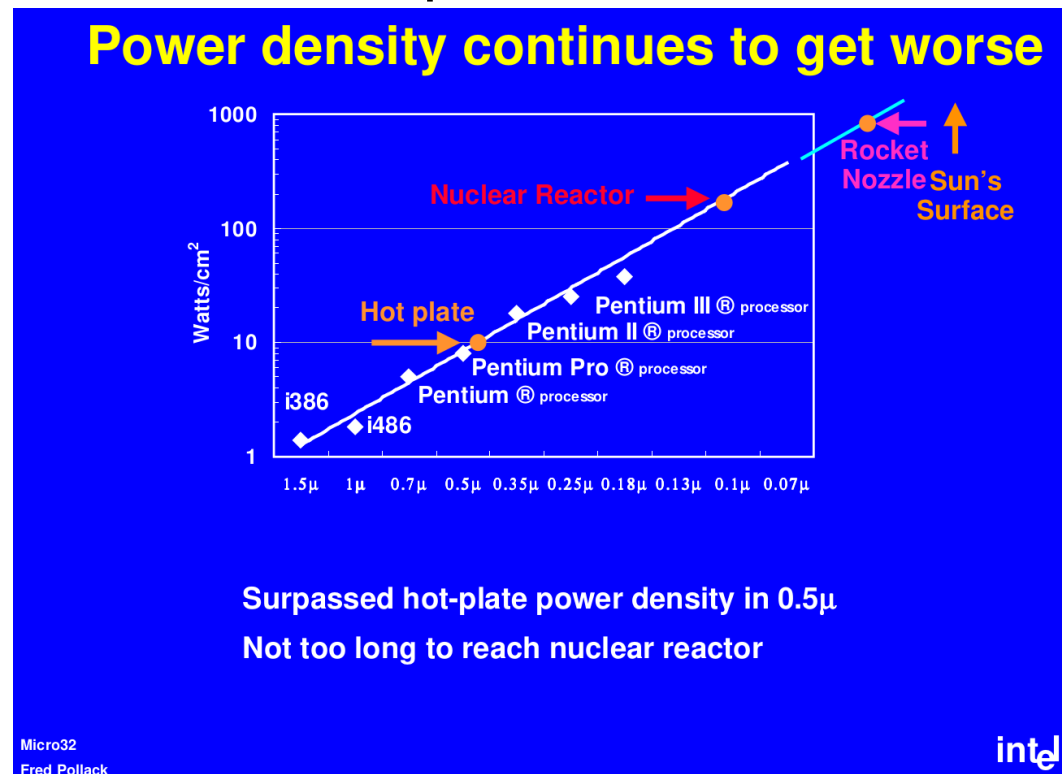
Moore, G.E., *Cramming more components onto integrated circuits*. Electronics, 38(8), April 1965



Gordon E.
Moore
(1929–)

Physics lesson

- Smaller transistors → Faster processor
- Faster processor → Higher power consumption
- Higher power consumption → More heat
- More heat → Unreliable processor



Power

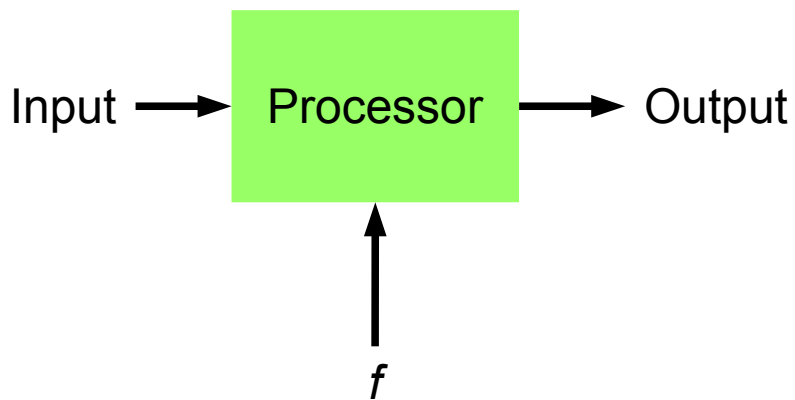
- The power required by an IC (e.g., a processor) can be expressed as

$$Power = C \times V^2 \times f$$

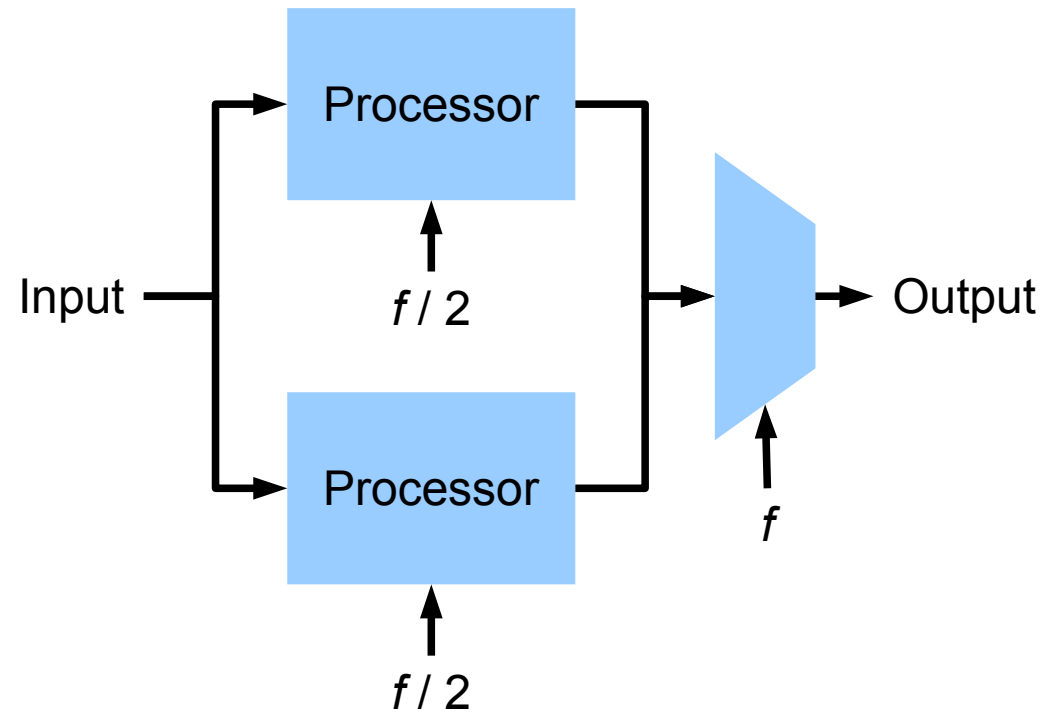
where:

- *C* is the *capacitance* (ability of a circuit to store energy)
- *V* is the *voltage*
- *f* is the *frequency* at which the processor operates

Power

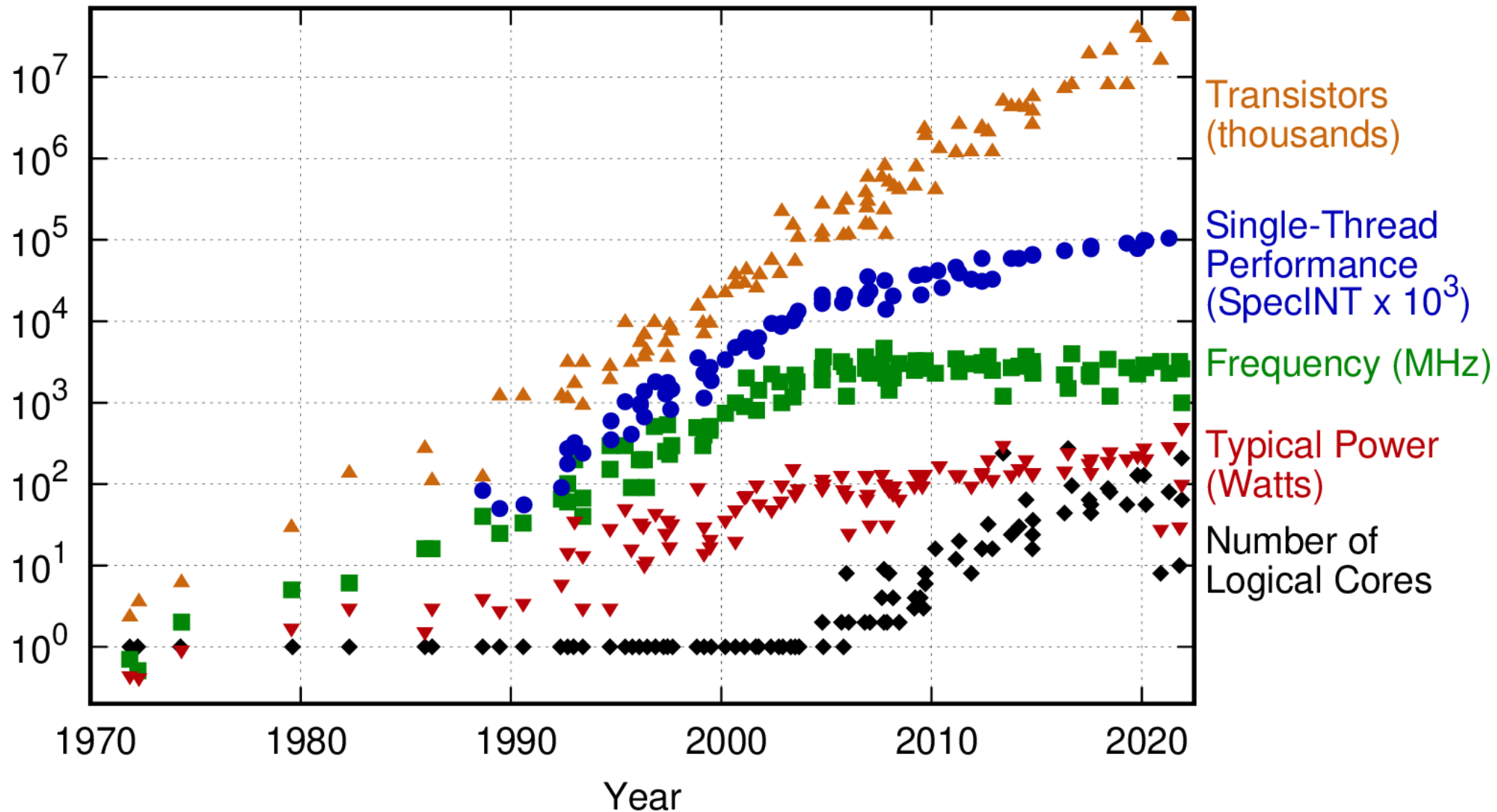


Capacitance C
Voltage V
Frequency f
Power = $C V^2 f$



Capacitance $2.2 C$
Voltage $0.6 V$
Frequency $0.5 f$
Power = $0.396 C V^2 f$

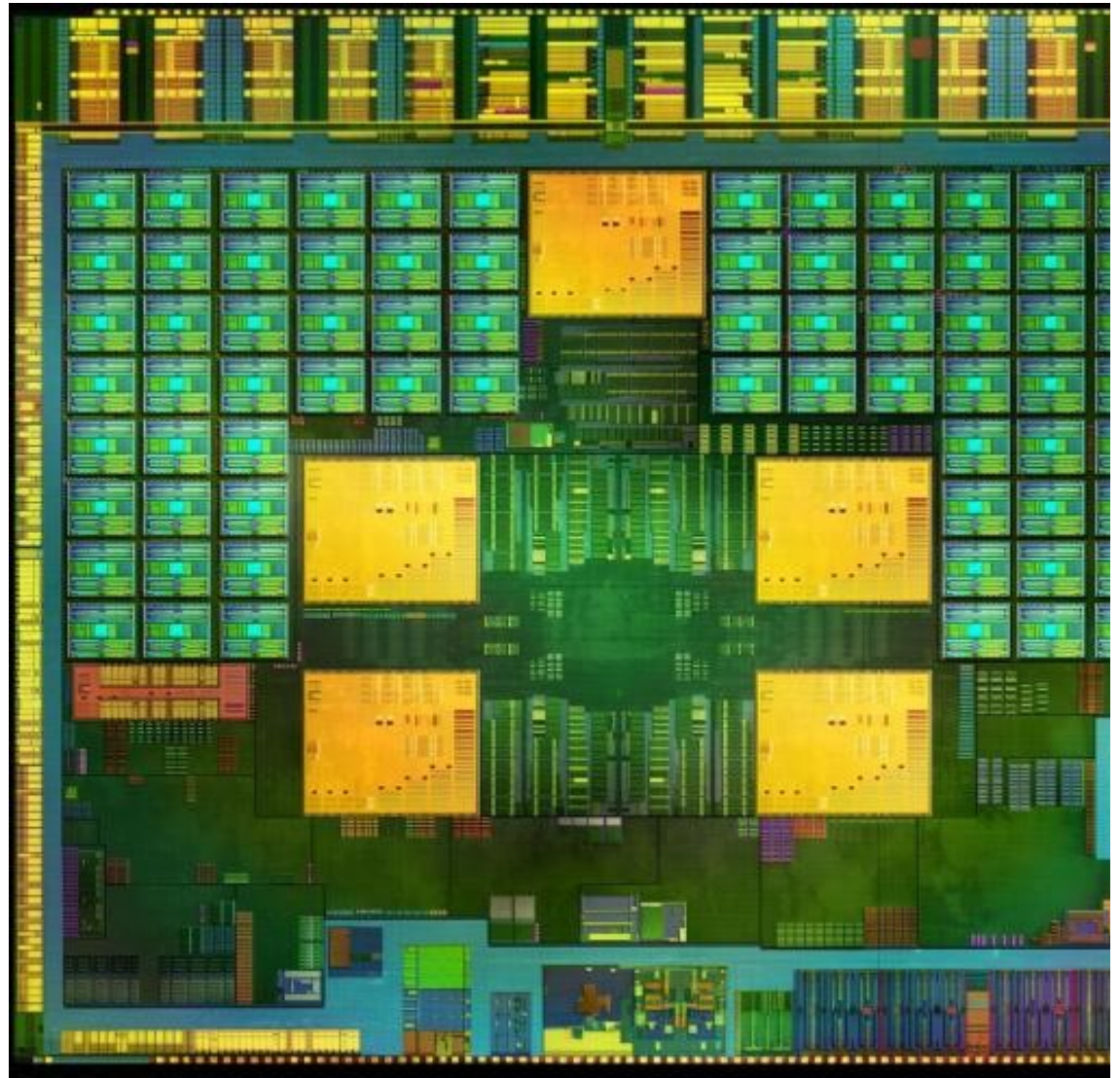
50 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

What happens today?

- HW designers create processors with more cores
- The result
 - Parallel hardware is **ubiquitous**
 - Parallel software is **rare**
- The challenge
 - Make parallel software as common as parallel hardware



Parallel programming in brief

- **Decompose** the problem in sub-problems
- **Distribute** sub-problems to the available execution units
- **Solve** sub-problems independently
 - **Cooperate** to solve sub-problems
- **Goals**
 - Reduce the wall-clock time
 - Balance the workload across execution units
 - Reduce communication and synchronization overhead

The “Holy Grail”

- Write serial code and have a “smart” compiler capable of parallelizing programs automatically
- It has been done in some very specific cases
 - In practice, no compiler proved to be “smart” enough
- Writing efficient parallel code requires that **the programmer** understands, and makes explicit use of, the underlying hardware  *Here we come*

<http://www.moreno.marzolla.name/software/svmcell/>

Parallel version (~1000 lines of C/C++ code)

Serial version (~49 lines of C++ code)

[illegible][illegible][illegible][illegible][illegible]

```

Printed by Mycroft ML
Page 10

pipps.py --cc
#
# See: 14-01-2016, 14-02-2016, 14-03-2016, 14-04-2016, 14-05-2016, 14-06-2016, 14-07-2016, 14-08-2016, 14-09-2016, 14-10-2016, 14-11-2016, 14-12-2016, 15-01-2016, 15-02-2016, 15-03-2016, 15-04-2016, 15-05-2016, 15-06-2016, 15-07-2016, 15-08-2016, 15-09-2016, 15-10-2016, 15-11-2016, 15-12-2016, 16-01-2016, 16-02-2016, 16-03-2016, 16-04-2016, 16-05-2016, 16-06-2016, 16-07-2016, 16-08-2016, 16-09-2016, 16-10-2016, 16-11-2016, 16-12-2016, 17-01-2016, 17-02-2016, 17-03-2016, 17-04-2016, 17-05-2016, 17-06-2016, 17-07-2016, 17-08-2016, 17-09-2016, 17-10-2016, 17-11-2016, 17-12-2016, 18-01-2016, 18-02-2016, 18-03-2016, 18-04-2016, 18-05-2016, 18-06-2016, 18-07-2016, 18-08-2016, 18-09-2016, 18-10-2016, 18-11-2016, 18-12-2016, 19-01-2016, 19-02-2016, 19-03-2016, 19-04-2016, 19-05-2016, 19-06-2016, 19-07-2016, 19-08-2016, 19-09-2016, 19-10-2016, 19-11-2016, 19-12-2016, 20-01-2016, 20-02-2016, 20-03-2016, 20-04-2016, 20-05-2016, 20-06-2016, 20-07-2016, 20-08-2016, 20-09-2016, 20-10-2016, 20-11-2016, 20-12-2016, 21-01-2016, 21-02-2016, 21-03-2016, 21-04-2016, 21-05-2016, 21-06-2016, 21-07-2016, 21-08-2016, 21-09-2016, 21-10-2016, 21-11-2016, 21-12-2016, 22-01-2016, 22-02-2016, 22-03-2016, 22-04-2016, 22-05-2016, 22-06-2016, 22-07-2016, 22-08-2016, 22-09-2016, 22-10-2016, 22-11-2016, 22-12-2016, 23-01-2016, 23-02-2016, 23-03-2016, 23-04-2016, 23-05-2016, 23-06-2016, 23-07-2016, 23-08-2016, 23-09-2016, 23-10-2016, 23-11-2016, 23-12-2016, 24-01-2016, 24-02-2016, 24-03-2016, 24-04-2016, 24-05-2016, 24-06-2016, 24-07-2016, 24-08-2016, 24-09-2016, 24-10-2016, 24-11-2016, 24-12-2016, 25-01-2016, 25-02-2016, 25-03-2016, 25-04-2016, 25-05-2016, 25-06-2016, 25-07-2016, 25-08-2016, 25-09-2016, 25-10-2016, 25-11-2016, 25-12-2016, 26-01-2016, 26-02-2016, 26-03-2016, 26-04-2016, 26-05-2016, 26-06-2016, 26-07-2016, 26-08-2016, 26-09-2016, 26-10-2016, 26-11-2016, 26-12-2016, 27-01-2016, 27-02-2016, 27-03-2016, 27-04-2016, 27-05-2016, 27-06-2016, 27-07-2016, 27-08-2016, 27-09-2016, 27-10-2016, 27-11-2016, 27-12-2016, 28-01-2016, 28-02-2016, 28-03-2016, 28-04-2016, 28-05-2016, 28-06-2016, 28-07-2016, 28-08-2016, 28-09-2016, 28-10-2016, 28-11-2016, 28-12-2016, 29-01-2016, 29-02-2016, 29-03-2016, 29-04-2016, 29-05-2016, 29-06-2016, 29-07-2016, 29-08-2016, 29-09-2016, 29-10-2016, 29-11-2016, 29-12-2016, 30-01-2016, 30-02-2016, 30-03-2016, 30-04-2016, 30-05-2016, 30-06-2016, 30-07-2016, 30-08-2016, 30-09-2016, 30-10-2016, 30-11-2016, 30-12-2016, 31-01-2016, 31-02-2016, 31-03-2016, 31-04-2016, 31-05-2016, 31-06-2016, 31-07-2016, 31-08-2016, 31-09-2016, 31-10-2016, 31-11-2016, 31-12-2016, 1-01-2017, 1-02-2017, 1-03-2017, 1-04-2017, 1-05-2017, 1-06-2017, 1-07-2017, 1-08-2017, 1-09-2017, 1-10-2017, 1-11-2017, 1-12-2017, 2-01-2017, 2-02-2017, 2-03-2017, 2-04-2017, 2-05-2017, 2-06-2017, 2-07-2017, 2-08-2017, 2-09-2017, 2-10-2017, 2-11-2017, 2-12-2017, 3-01-2017, 3-02-2017, 3-03-2017, 3-04-2017, 3-05-2017, 3-06-2017, 3-07-2017, 3-08-2017, 3-09-2017, 3-10-2017, 3-11-2017, 3-12-2017, 4-01-2017, 4-02-2017, 4-03-2017, 4-04-2017, 4-05-2017, 4-06-2017, 4-07-2017, 4-08-2017, 4-09-2017, 4-10-2017, 4-11-2017, 4-12-2017, 5-01-2017, 5-02-2017, 5-03-2017, 5-04-2017, 5-05-2017, 5-06-2017, 5-07-2017, 5-08-2017, 5-09-2017, 5-10-2017, 5-11-2017, 5-12-2017, 6-01-2017, 6-02-2017, 6-03-2017, 6-04-2017, 6-05-2017, 6-06-2017, 6-07-2017, 6-08-2017, 6-09-2017, 6-10-2017, 6-11-2017, 6-12-2017, 7-01-2017, 7-02-2017, 7-03-2017, 7-04-2017, 7-05-2017, 7-06-2017, 7-07-2017, 7-08-2017, 7-09-2017, 7-10-2017, 7-11-2017, 7-12-2017, 8-01-2017, 8-02-2017, 8-03-2017, 8-04-2017, 8-05-2017, 8-06-2017, 8-07-2017, 8-08-2017, 8-09-2017, 8-10-2017, 8-11-2017, 8-12-2017, 9-01-2017, 9-02-2017, 9-03-2017, 9-04-2017, 9-05-2017, 9-06-2017, 9-07-2017, 9-08-2017, 9-09-2017, 9-10-2017, 9-11-2017, 9-12-2017, 10-01-2017, 10-02-2017, 10-03-2017, 10-04-2017, 10-05-2017, 10-06-2017, 10-07-2017, 10-08-2017, 10-09-2017, 10-10-2017, 10-11-2017, 10-12-2017, 11-01-2017, 11-02-2017, 11-03-2017, 11-04-2017, 11-05-2017, 11-06-2017, 11-07-2017, 11-08-2017, 11-09-2017, 11-10-2017, 11-11-2017, 11-12-2017, 1
```

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

```

Purified by Means: 100.00
Page: 10/10

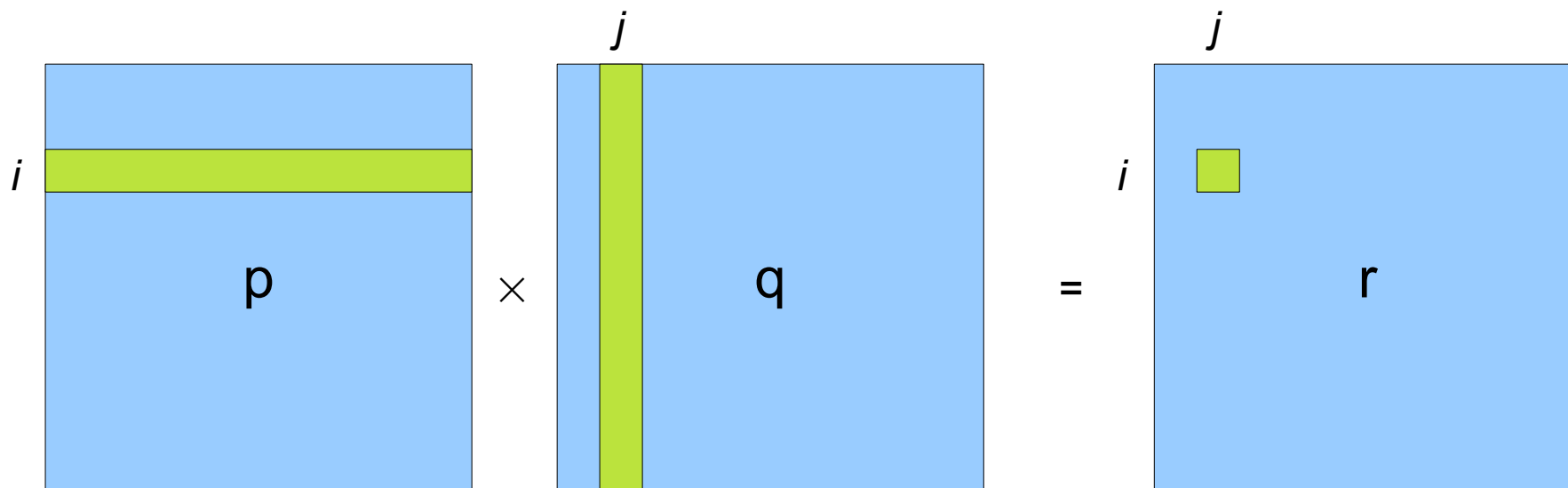
1  // 1
2  // 2
3  // 3
4  // 4
5  // 5
6  // 6
7  // 7
8  // 8
9  // 9
10 // 10
11 // 11
12 // 12
13 // 13
14 // 14
15 // 15
16 // 16
17 // 17
18 // 18
19 // 19
20 // 20
21 // 21
22 // 22
23 // 23
24 // 24
25 // 25
26 // 26
27 // 27
28 // 28
29 // 29
30 // 30
31 // 31
32 // 32
33 // 33
34 // 34
35 // 35
36 // 36
37 // 37
38 // 38
39 // 39
40 // 40
41 // 41
42 // 42
43 // 43
44 // 44
45 // 45
46 // 46
47 // 47
48 // 48
49 // 49
50 // 50
51 // 51
52 // 52
53 // 53
54 // 54
55 // 55
56 // 56
57 // 57
58 // 58
59 // 59
60 // 60
61 // 61
62 // 62
63 // 63
64 // 64
65 // 65
66 // 66
67 // 67
68 // 68
69 // 69
70 // 70
71 // 71
72 // 72
73 // 73
74 // 74
75 // 75
76 // 76
77 // 77
78 // 78
79 // 79
80 // 80
81 // 81
82 // 82
83 // 83
84 // 84
85 // 85
86 // 86
87 // 87
88 // 88
89 // 89
90 // 90
91 // 91
92 // 92
93 // 93
94 // 94
95 // 95
96 // 96
97 // 97
98 // 98
99 // 99
100 // 100
101 // 101
102 // 102
103 // 103
104 // 104
105 // 105
106 // 106
107 // 107
108 // 108
109 // 109
110 // 110
111 // 111
112 // 112
113 // 113
114 // 114
115 // 115
116 // 116
117 // 117
118 // 118
119 // 119
120 // 120
121 // 121
122 // 122
123 // 123
124 // 124
125 // 125
126 // 126
127 // 127
128 // 128
129 // 129
130 // 130
131 // 131
132 // 132
133 // 133
134 // 134
135 // 135
136 // 136
137 // 137
138 // 138
139 // 139
140 // 140
141 // 141
142 // 142
143 // 143
144 // 144
145 // 145
146 // 146
147 // 147
148 // 148
149 // 149
150 // 150
151 // 151
152 // 152
153 // 153
154 // 154
155 // 155
156 // 156
157 // 157
158 // 158
159 // 159
160 // 160
161 // 161
162 // 162
163 // 163
164 // 164
165 // 165
166 // 166
167 // 167
168 // 168
169 // 169
170 // 170
171 // 171
172 // 172
173 // 173
174 // 174
175 // 175
176 // 176
177 // 177
178 // 178
179 // 179
180 // 180
181 // 181
182 // 182
183 // 183
184 // 184
185 // 185
186 // 186
187 // 187
188 // 188
189 // 189
190 // 190
191 // 191
192 // 192
193 // 193
194 // 194
195 // 195
196 // 196
197 // 197
198 // 198
199 // 199
200 // 200
201 // 201
202 // 202
203 // 203
204 // 204
205 // 205
206 // 206
207 // 207
208 // 208
209 // 209
210 // 210
211 // 211
212 // 212
213 // 213
214 // 214
215 // 215
216 // 216
217 // 217
218 // 218
219 // 219
220 // 220
221 // 221
222 // 222
223 // 223
224 // 224
225 // 225
226 // 226
227 // 227
228 // 228
229 // 229
230 // 230
231 // 231
232 // 232
233 // 233
234 // 234
235 // 235
236 // 236
237 // 237
238 // 238
239 // 239
240 // 240
241 // 241
242 // 242
243 // 243
244 // 244
245 // 245
246 // 246
247 // 247
248 // 248
249 // 249
250 // 250
251 // 251
252 // 252
253 // 253
254 // 254
255 // 255
256 // 256
257 // 257
258 // 258
259 // 259
260 // 260
261 // 261
262 // 262
263 // 263
264 // 264
265 // 265
266 // 266
267 // 267
268 // 268
269 // 269
270 // 270
271 // 271
272 // 272
273 // 273
274 // 274
275 // 275
276 // 276
277 // 277
278 // 278
279 // 279
280 // 280
281 // 281
282 // 282
283 // 283
284 // 284
285 // 285
286 // 286
287 // 287
288 // 288
289 // 289
290 // 290
291 // 291
292 // 292
293 // 293
294 // 294
295 // 295
296 // 296
297 // 297
298 // 298
299 // 299
300 // 300
301 // 301
302 // 302
303 // 303
304 // 304
305 // 305
306 // 306
307 // 307
308 // 308
309 // 309
310 // 310
311 // 311
312 // 312
313 // 313
314 // 314
315 // 315
316 // 316
317 // 317
318 // 318
319 // 319
320 // 320
321 // 321
322 // 322
323 // 323
324 // 324
325 // 325
326 // 326
327 // 327
328 // 328
329 // 329
330 // 330
331 // 331
332 // 332
333 // 333
334 // 334
335 // 335
336 // 336
337 // 337
338 // 338
339 // 339
340 // 340
341 // 341
342 // 342
343 // 343
344 // 344
345 // 345
346 // 346
347 // 347
348 // 348
349 // 349
350 // 350
351 // 351
352 // 352
353 // 353
354 // 354
355 // 355
356 // 356
357 // 357
358 // 358
359 // 359
360 // 360
361 // 361
362 // 362
363 // 363
364 // 364
365 // 365
366 // 366
367 // 367
368 // 368
369 // 369
370 // 370
371 // 371
372 // 372
373 // 373
374 // 374
375 // 375
376 // 376
377 // 377
378 // 378
379 // 379
380 // 380
381 // 381
382 // 382
383 // 383
384 // 384
385 // 385
386 // 386
387 // 387
388 // 388
389 // 389
390 // 390
391 // 391
392 // 392
393 // 393
394 // 394
395 // 395
396 // 396
397 // 397
398 // 398
399 // 399
400 // 400
401 // 401
402 // 402
403 // 403
404 // 404
405 // 405
406 // 406
407 // 407
408 // 408
409 // 409
410 // 410
411 // 411
412 // 412
413 // 413
414 // 414
415 // 415
416 // 416
417 // 417
418 // 418
419 // 419
420 // 420
421 // 421
422 // 422
423 // 423
424 // 424
425 // 425
426 // 426
427 // 427
428 // 428
429 // 429
430 // 430
431 // 431
432 // 432
433 // 433
434 // 434
435 // 435
436 // 436
437 // 437
438 // 438
439 // 439
440 // 440
441 // 441
442 // 442
443 // 443
444 // 444
445 // 445
446 // 446
447 // 447
448 // 448
449 // 449
450 // 450
451 // 451
452 // 452
453 // 453
454 // 454
455 // 455
456 // 456
457 // 457
458 // 458
459 // 459
460 // 460
461 // 461
462 // 462
463 // 463
464 // 464
465 // 465
466 // 466
467 // 467
468 // 468
469 // 469
470 // 470
471 // 471
472 // 472
47
```

Issues of parallel programming

- Writing parallel programs is in general **much harder** than writing sequential code
- There is **limited portability** across **different types** of architectures
 - E.g., a distributed-memory parallel program must be rewritten from scratch to run on a GPU
 - However, there are standards (OpenMP, MPI, OpenCL) that allow portability across the **same type** of parallel architecture
- **Tuning** for best performance is time-consuming

A case study in dense matrix-matrix multiplication

Matmul



$$r_{ij} = \sum_{k=0}^{n-1} p_{ik} q_{kj}$$

Hardware

Feature	Specification
Processor type	Intel(R) Core(TM) i7-9800X
Clock frequency	3.80 GHz
Processor chips	1
Processing cores	8 (physical)
Hyperthreading	2 way (→ 16 “virtual” cores)
Floating-point unit	742 Gflops
L1-icache	32 KB × 8
L1-dcache	32 KB × 8
L2-cache	1 MB × 8
L3-cache	16 MB (shared)
DRAM	32 GB
Python	3.10.12
gcc	11.4.0, <code>-std=c99 -Wall -Wpedantic -fopenmp -mavx2 -mfma -O2</code>
Java	11.0.24

Version 1: Python

```
# matmul-test.py
import sys, random

n = 4096

p = [[random.random() for row in range(n)]
      for col in range(n)]
q = [[random.random() for row in range(n)]
      for col in range(n)]
r = [[0 for row in range(n)]
      for col in range(n)]

start = time()
for i in range(n):
    for j in range(n):
        for k in range(n):
            r[i][j] += p[i][k] * q[k][j]
elapsed = time() - start
print(elapsed)
```

Running time
= **19617** seconds
= **5.4** hours

$2n^3$ floating-point
operations in
19617s
≈ 7 MFLOPS

Peak **≈ 742**
GFLOPS

Python gets
0.0009% of peak

Version 2: Java

```
import java.util.Random;
import java.io.*;
public class matmul_test {
    final static int n = 4096;
    static double[][] p = new double[n][n];
    static double[][] q = new double[n][n];
    static double[][] r = new double[n][n];
    public static void main(String[] args) {
        Random rnd = new Random();
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                p[i][j] = rnd.nextDouble();
                q[i][j] = rnd.nextDouble();
                r[i][j] = 0;
            }
        }
        final long start = System.nanoTime();
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                for (int k=0; k<n; k++)
                    r[i][j] += p[i][k] * q[k][j];
        final long stop = System.nanoTime();
        System.out.println((stop - start)*1e-9);
    }
}
```

Running time
= **838** seconds
= **13,9** minutes

23 times faster
than Python

Version 3: C

```
void matmul(const float* p,
            const float* q,
            float *r, int n)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            r[i*n + j] = 0;

    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            for (int k=0; k<n; k++)
                r[i*n + j] += p[i*n + k] * q[k*n + j];
}
```

Running time
= **595** seconds
= **9,9** minutes

33 times faster
than Python

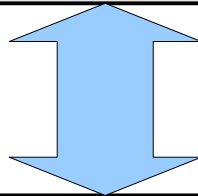
Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06

Reordering loops

- In this particular examples, the three nested loops can be permuted and still produce the same result

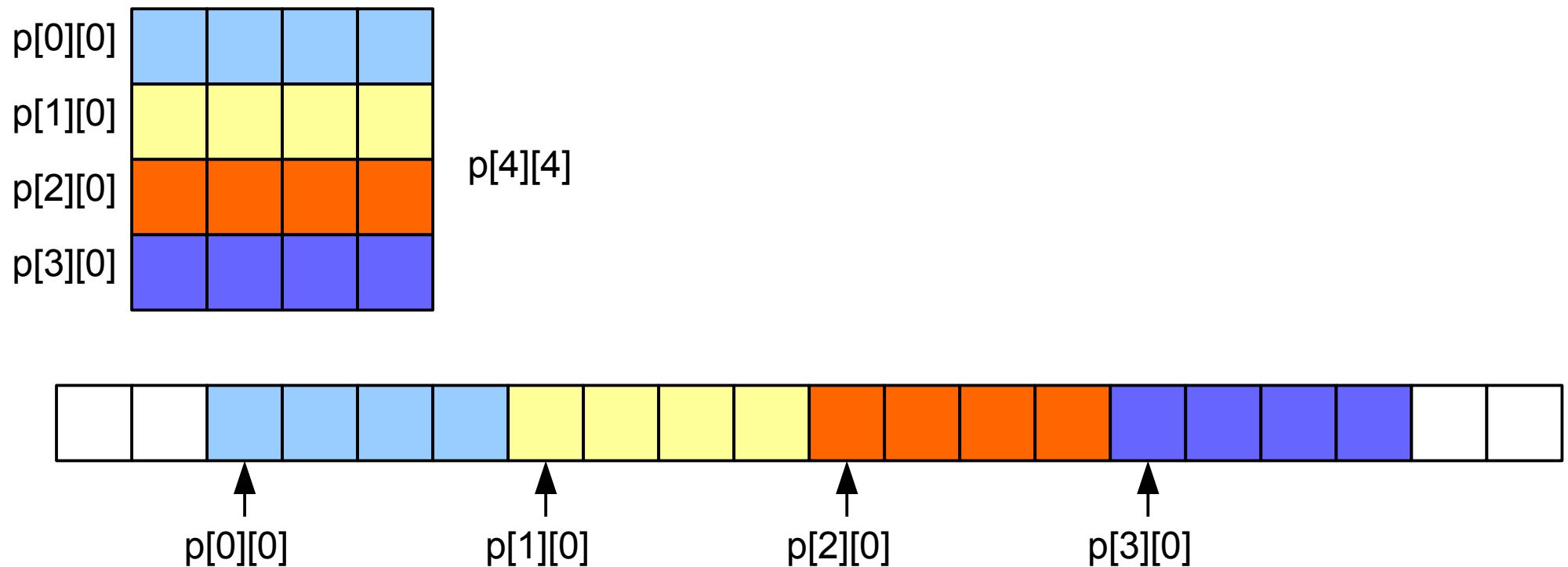
```
for (int i=0; i<n; i++)  
  for (int j=0; j<n; j++)  
    for (int k=0; k<n; k++)  
      r[i*n+j] += p[i*n+k] * q[k*n+j];
```



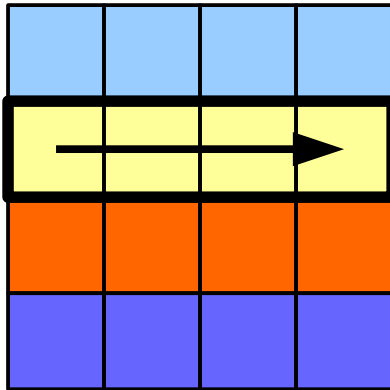
```
for (int k=0; k<n; k++)  
  for (int i=0; i<n; i++)  
    for (int j=0; j<n; j++)  
      r[i*n+j] += p[i*n+k] * q[k*n+j];
```

Matrix representation

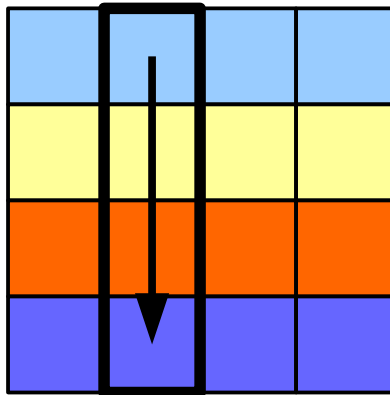
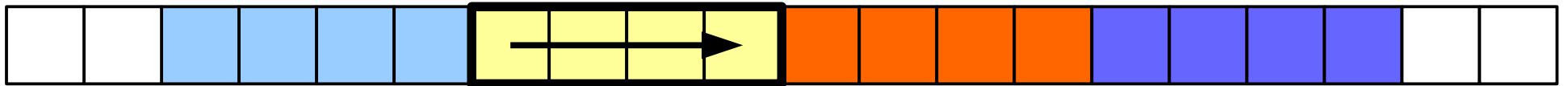
- Matrices in C are stored in *row-major order*
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory



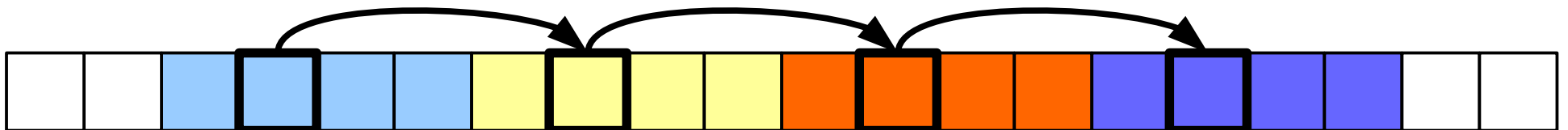
Row-wise vs column-wise access



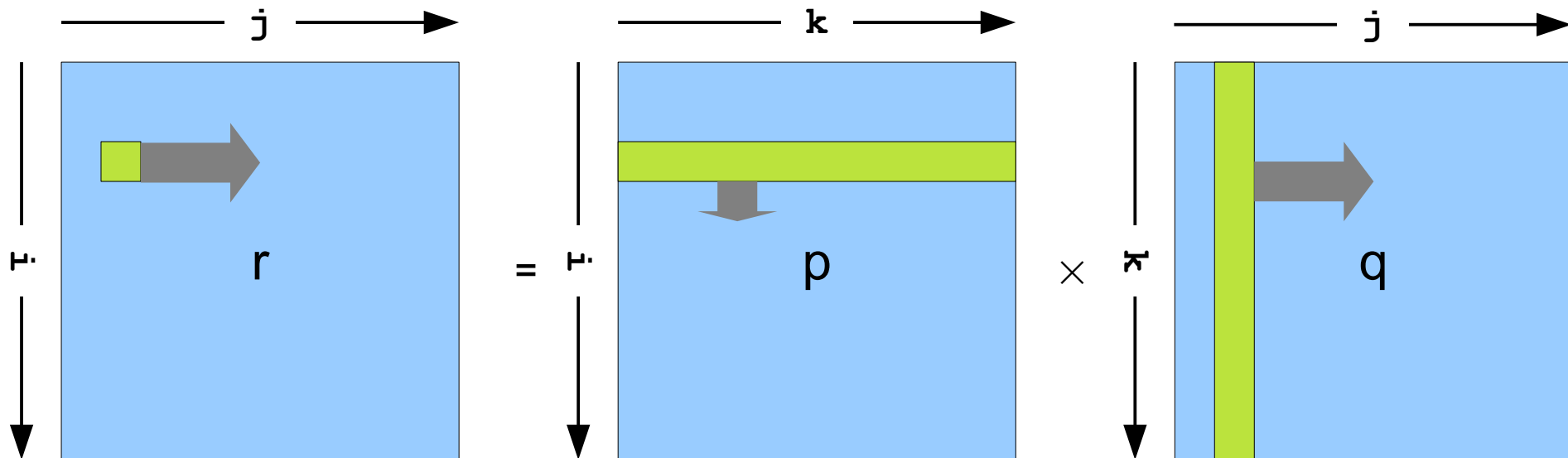
Row-wise access is OK: the data is contiguous in memory, so the cache helps (spatial locality)



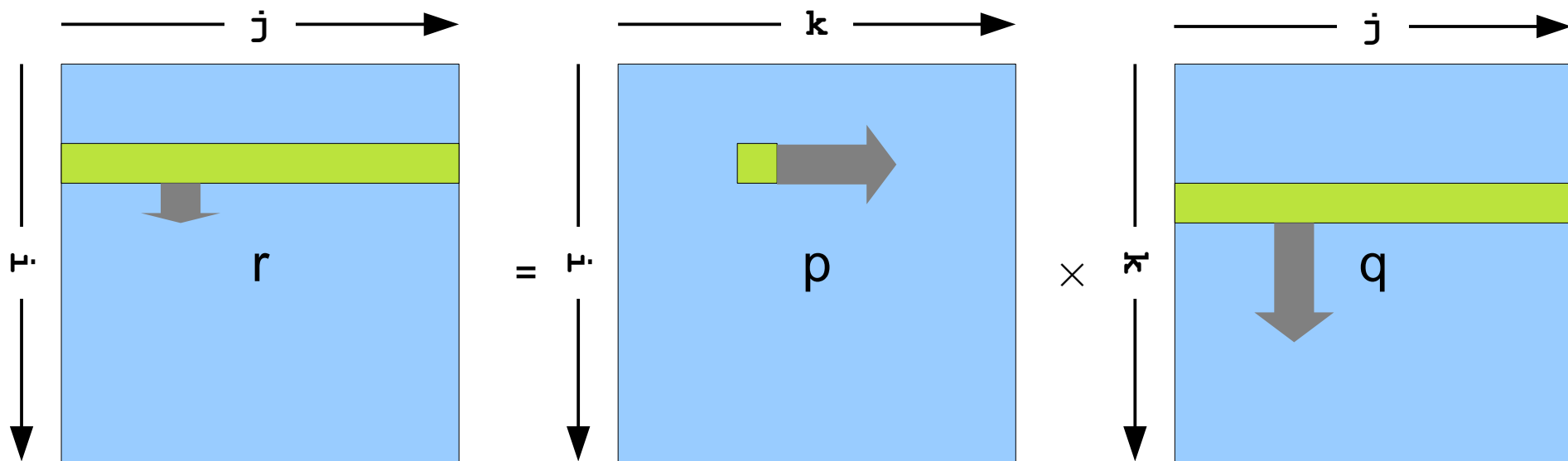
Column-wise access is **NOT OK**: the accessed elements are not contiguous in memory (*strided* access) so the cache does NOT help



i, j, k



i, k, j



Reordering loops

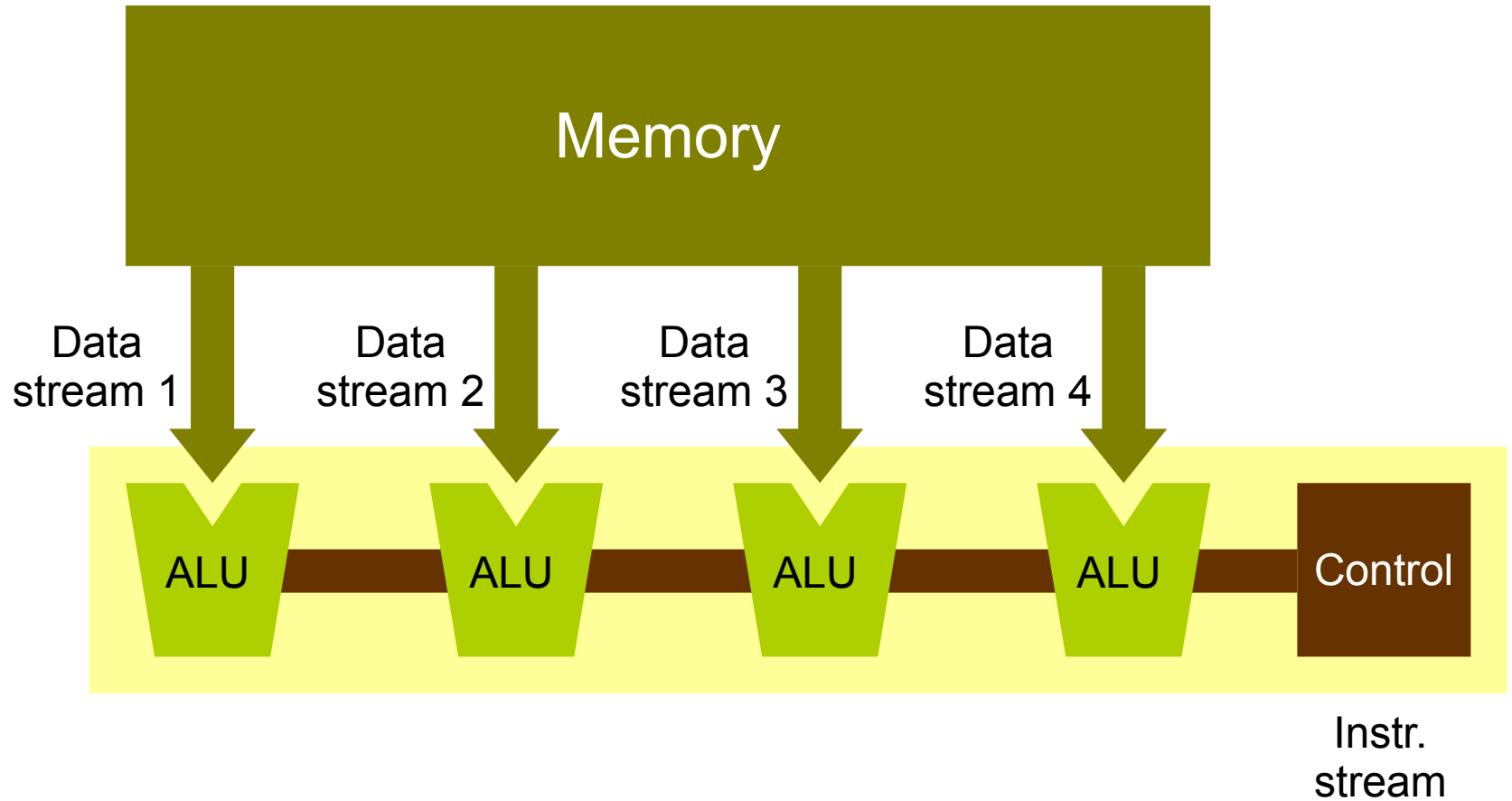
Permutation	Time (s)	GFLOPS
i, j, k	595,85	0,23
i, k, j	32,06	4,29
j, i, k	343,08	0,40
j, k, i	824,92	0,17
k, i, j	36,65	3,75
k, j, i	1097,55	0,13

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06
+ Loop reordering	32.06	4.29	0.58%	611.93

Single Instruction Multiple Data

SIMD instructions



Parallelizable $256 / 32 = 8$ ops

=> 256 is intel register bits, as found in machine dependant operations => these functions make us loose the portability

=> 32 is the data size, which is float

```
void matmul_simd_ikj( const float *p, const float *q,
                      float *r, int n)
{
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j += VLEN) {
            __m256 *ptr = (__m256*)&r[i*n + j];
            *ptr = _mm256_setzero_ps();
        }
    }

    for (int i=0; i<n; i++) {
        for (int k=0; k<n; k++) {
            const __m256 pv = _mm256_broadcast_ss(&p[i*n + k]);
            for (int j=0; j<n; j += VLEN) {
                __m256 rv = _mm256_load_ps( &r[i*n + j] );
                const __m256 qv = _mm256_load_ps( &q[k*n + j] );
                rv = _mm256_fmadd_ps(pv, qv, rv);
                _mm256_store_ps( &r[i*n + j], rv );
            }
        }
    }
}
```

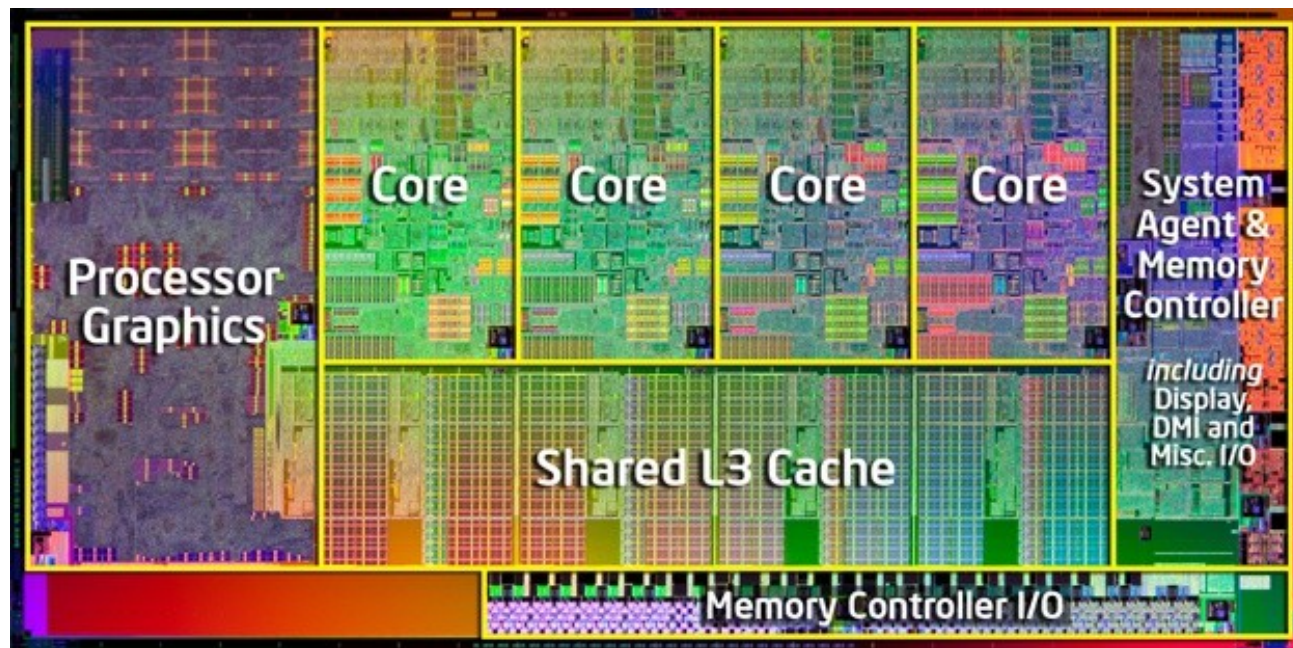
NB: this requires that the matrix size **n** is multiple of **VLEN**

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06
+ Loop reordering	32.06	4.29	0.58%	611.93
+ SIMD	19.37	7.09	0.96%	1012.66

Multicore parallelism

- Modern processors contain multiple independent execution units (“cores”)
- The machine used for the tests has 8 of them
 - Each one is seen by the OS as two “virtual” cores
 - Virtual cores share some physical resources and are therefore less powerful than “true” physical cores



Instructions for the compiler, they are the closest solution so the “smart compiler” that handles automatically parallelization. But Here we have to say it how to parallelize and what but it parallelizes it then automatically

```
void matmul_simd_ikj( const float *p, const float *q,
                    float *r, int n) {
    #pragma omp parallel default(none) shared(p,q,r,n)
    {
        #pragma omp for
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j += VLEN) {
                __m256 *ptr = (__m256*)&r[i*n + j];
                *ptr = _mm256_setzero_ps();
            }
        }
        #pragma omp for
        for (int i=0; i<n; i++) {
            for (int k=0; k<n; k++) {
                const __m256 pv = _mm256_broadcast_ss(&p[i*n + k]);
                for (int j=0; j<n; j += VLEN) {
                    __m256 rv = _mm256_load_ps( &r[i*n + j] );
                    const __m256 qv = _mm256_load_ps( &q[k*n + j] );
                    rv = _mm256_fmadd_ps(pv, qv, rv);
                    _mm256_store_ps( &r[i*n + j], rv );
                }
            }
        }
    }
}
```

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06
+ Loop reordering	32.06	4.29	0.58%	611.93
+ SIMD	19.37	7.09	0.96%	1012.66
+ OpenMP	1.76	77.96	10.51%	11127.21

Parallel programs design

Sum-Reduction

- We start assuming a **shared-memory architecture**
 - All execution units share a common memory space
- We begin with a sequential solution and parallelize it
 - This is not always a good idea; some parallel algorithms have nothing in common with their sequential counterparts!
 - However, it is sometimes a reasonable starting point

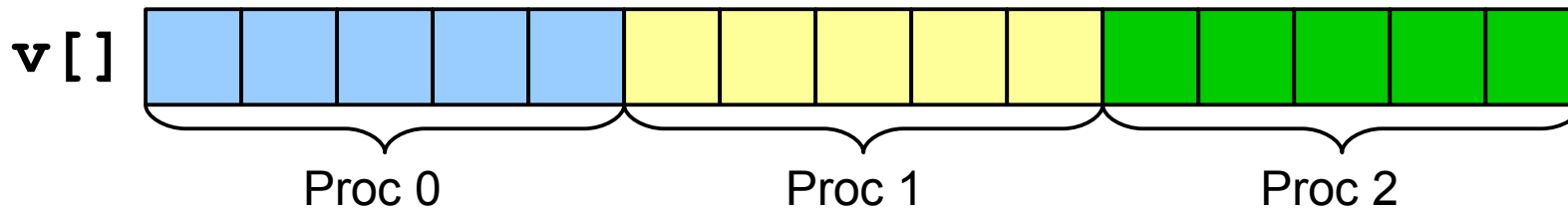
Sequential algorithm

- Compute the sum of the content of an array $v[]$ of length n

```
float seq_sum(const float* v, int n)
{
    int i;
    float sum = 0.0;
    for (i=0; i<n; i++) {
        sum += v[i];
    }
    return sum;
}
```

Version 1 Shared memory (Wrong!)

- Assuming P execution units (e.g., processors), each one computes a partial sum of n / P adjacent elements
- Example: $n = 15$, $P = 3$



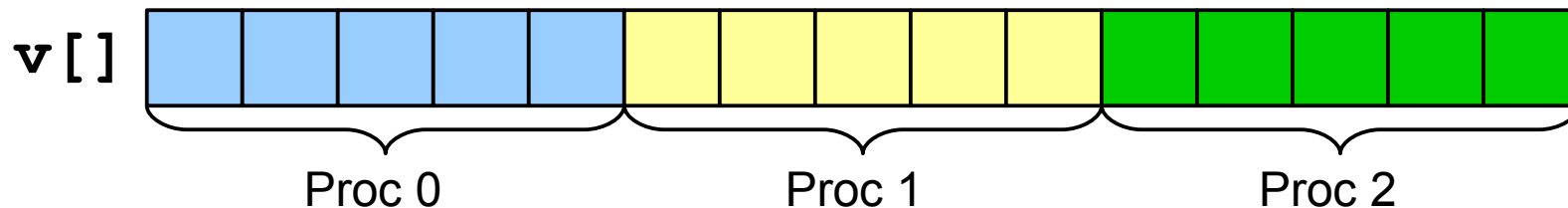
```
sum = 0.0;
do in parallel {
  my_block_len = n/P;
  my_start = my_id * my_block_len;
  my_end = my_start + my_block_len;
  for (my_i=my_start; my_i<my_end; my_i++) {
    sum += v[my_i];
  }
}
```

WRONG

Variables whose names start with `my_` are assumed to be **local** (private) to each processor; all other variables are assumed to be **global** (shared)

Version 1 (better, but still wrong)

- Assuming P processors, each one computes a partial sum of n/P adjacent elements
- Example: $n = 15$, $P = 3$

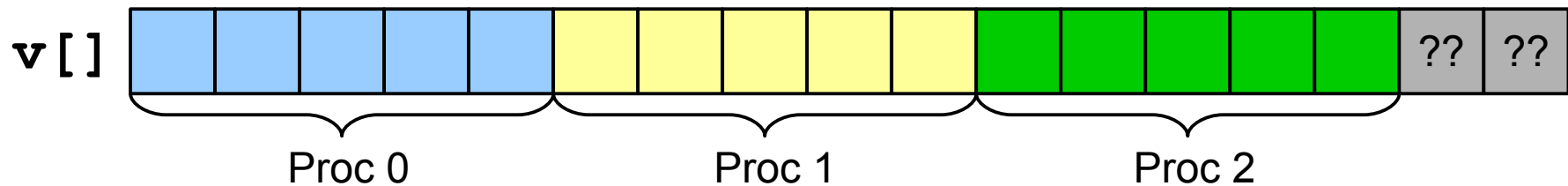


```
mutex m;  
sum = 0.0;  
do in parallel {  
    my_block_len = n/P;  
    my_start = my_id * my_block_len;  
    my_end = my_start + my_block_len;  
    for (my_i=my_start; my_i<my_end; my_i++) {  
        mutex_lock(&m);  
        sum += v[my_i];  
        mutex_unlock(&m);  
    }  
}
```

WRONG

Version 1 (better, but still wrong)

- Assuming P processors, each one computes a partial sum of n/P adjacent elements
- Example: $n = 17$, $P = 3$

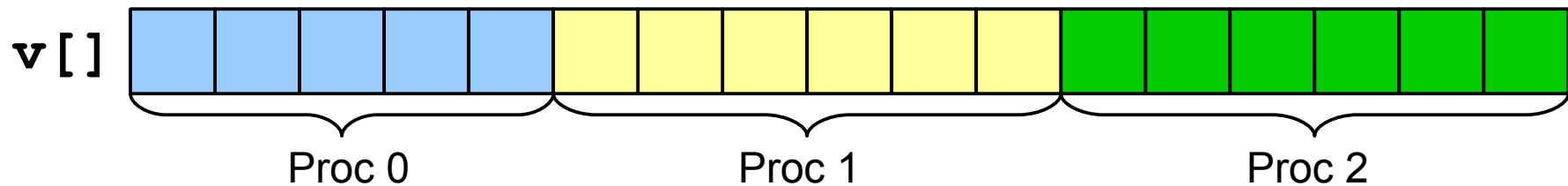


```
mutex m;  
sum = 0.0;  
do in parallel {  
    my_block_len = n/P;  
    my_start = my_id * my_block_len;  
    my_end = my_start + my_block_len;  
    for (my_i=my_start; my_i<my_end; my_i++) {  
        mutex_lock(&m);  
        sum += v[my_i];  
        mutex_unlock(&m);  
    }  
}
```

WRONG

Version 1 (correct, not efficient)

- Assuming P processors, each one computes a partial sum of n / P adjacent elements
- Example: $n = 17$, $P = 3$



```
mutex m;
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    for (my_i=my_start; my_i<my_end; my_i++) {
        mutex_lock(&m);
        sum += v[my_i];
        mutex_unlock(&m);
    }
}
```

Version 2

- Too much contention on mutex *m*
- Solution: increase the mutex *granularity*
 - Each processor accumulates the partial sum on a local (*private*) variable
 - The mutex is used at the end to update the *global* sum

```
mutex m; sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    my_sum = 0.0;
    for (my_i=my_start; my_i<my_end; my_i++) {
        my_sum += v[my_i];
    }
    mutex_lock(&m);
    sum += my_sum;
    mutex_unlock(&m);
}
```

Version 3: Remove the mutex

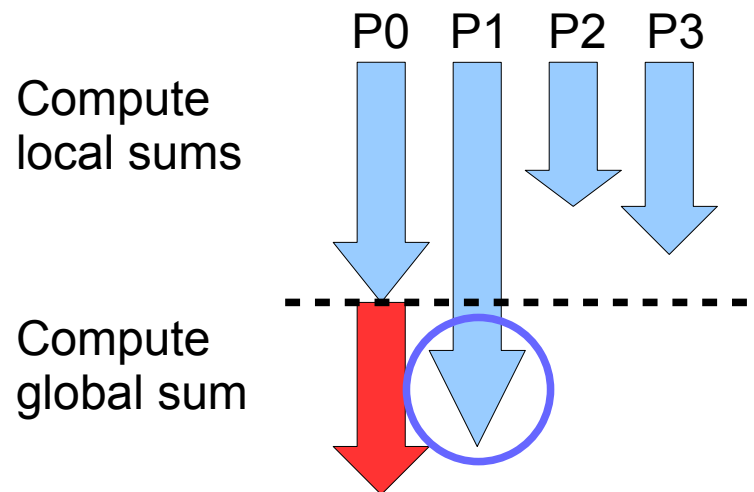
(wrong, in a subtle way)

- We use a shared array `psum[]` where each processor can store its local sum
- At the end, one processor computes the global sum

```
psum[0..P-1] = 0.0; /* all elements set to 0.0 */ WRONG
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    for (my_i=my_start; my_i<my_end; my_i++) {
        psum[my_id] += v[my_i];
    }
    if ( 0 == my_id ) { /* only the master executes this */
        for (my_i=0; my_i<P; my_i++)
            sum += psum[my_i];
    }
}
```

The problem with version 3

- Processor 0 could start the computation of the global sum before all other processors have computed the local sums!



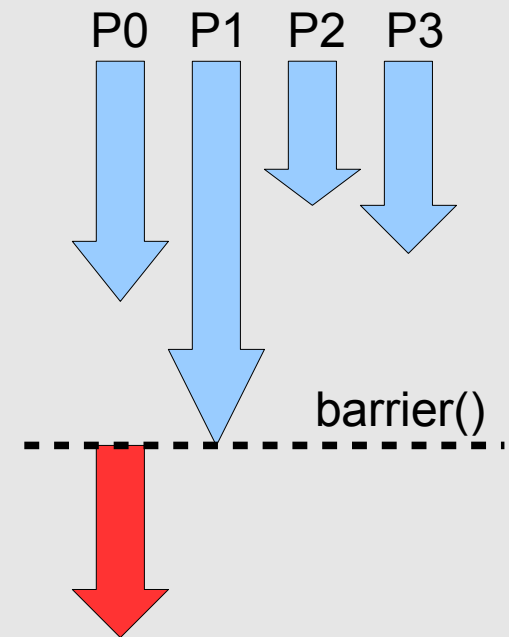
Version 4 (correct)

- Use a **barrier synchronization**

```
psum[0..P-1] = 0.0;
sum = 0.0;
do in parallel {
    my_start = (n * my_id) / P;
    my_end = (n * (my_id + 1)) / P;
    psum[0..P-1] = 0.0;
    for ( my_i=my_start;
          my_i<my_end; my_i++ ) {
        psum[my_id] += v[my_i];
    }
    barrier();
    for (my_i=0; my_i<P; my_i++)
        sum += psum[my_i];
}
```

Compute
local sums

Compute
global sum



Note: in OpenMP there is an implicit barrier at the end of a parallel region, so this pseudocode will be greatly simplified

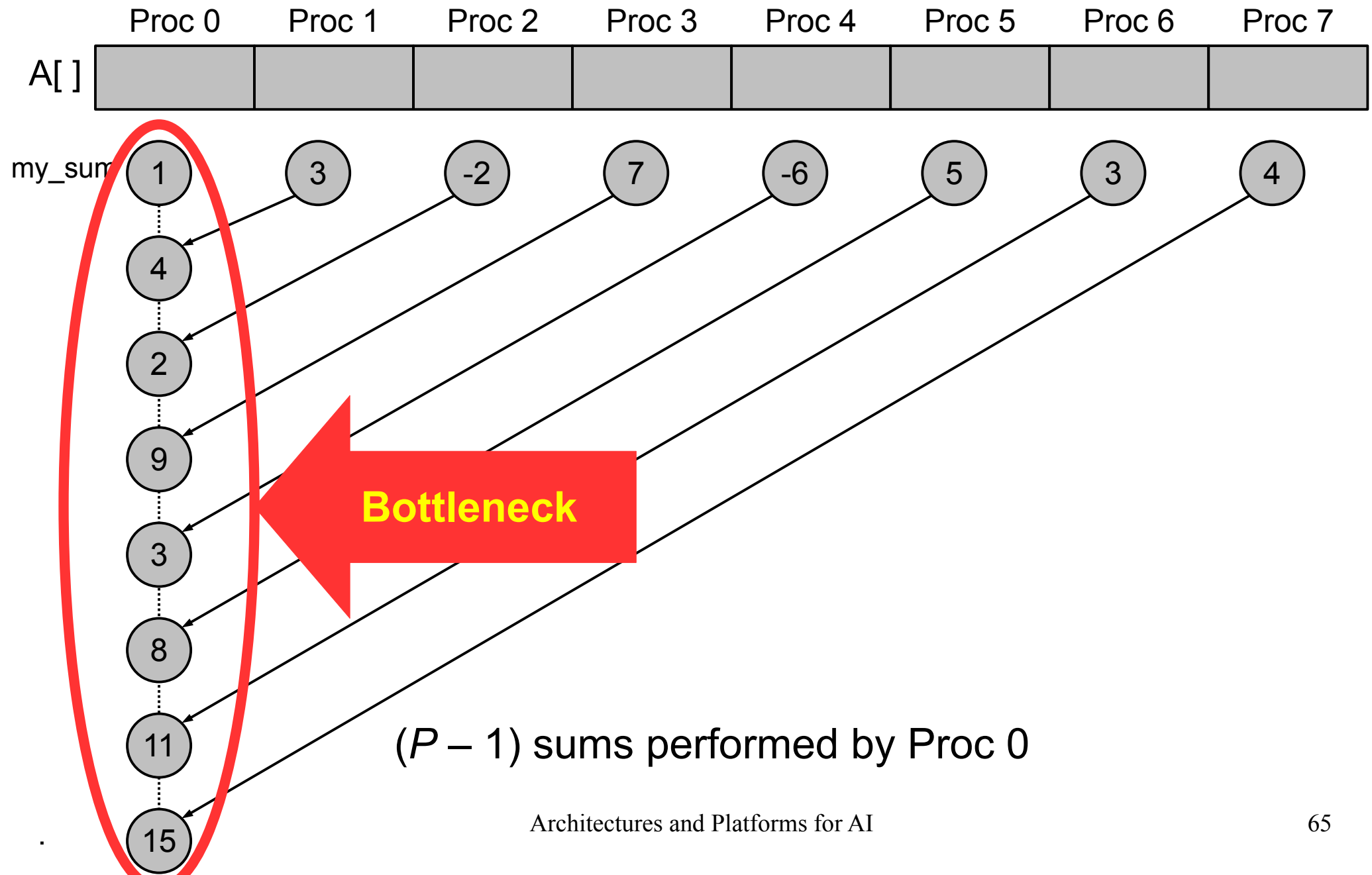
Version 5

Distributed-memory version

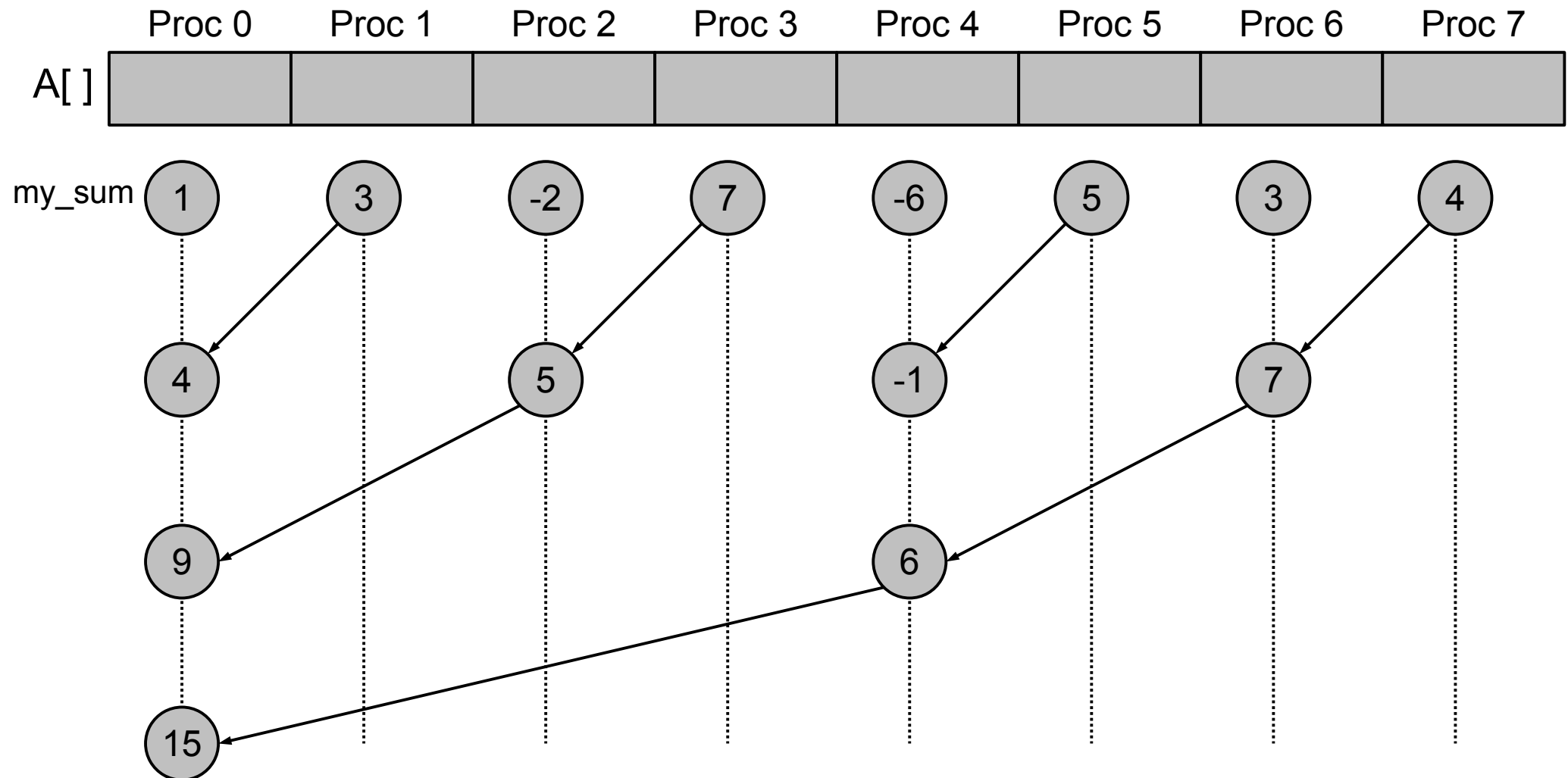
- $P \ll n$ processors
- Each processor computes a local sum
- Each processor sends the local sum to processor 0 (the master)

```
...
my_sum = 0.0;
my_start = ..., my_end = ...;
my_v = receive v[my_start..my_end-1]
from proc 0;
for (i=0; i < my_end-my_start; i++) {
    my_sum += my_v[i];
}
if ( 0 == my_id ) {
    for ( i=1; i < P; i++ ) {
        tmp = receive from proc i;
        my_sum += tmp;
    }
    printf("The sum is %f\n", my_sum);
} else {
    send my_sum to proc 0;
}
```

Version 5



Parallel reduction



- $(P - 1)$ sums are still performed; however, processor 0 receives $\sim \log_2 P$ messages and performs $\sim \log_2 P$ sums

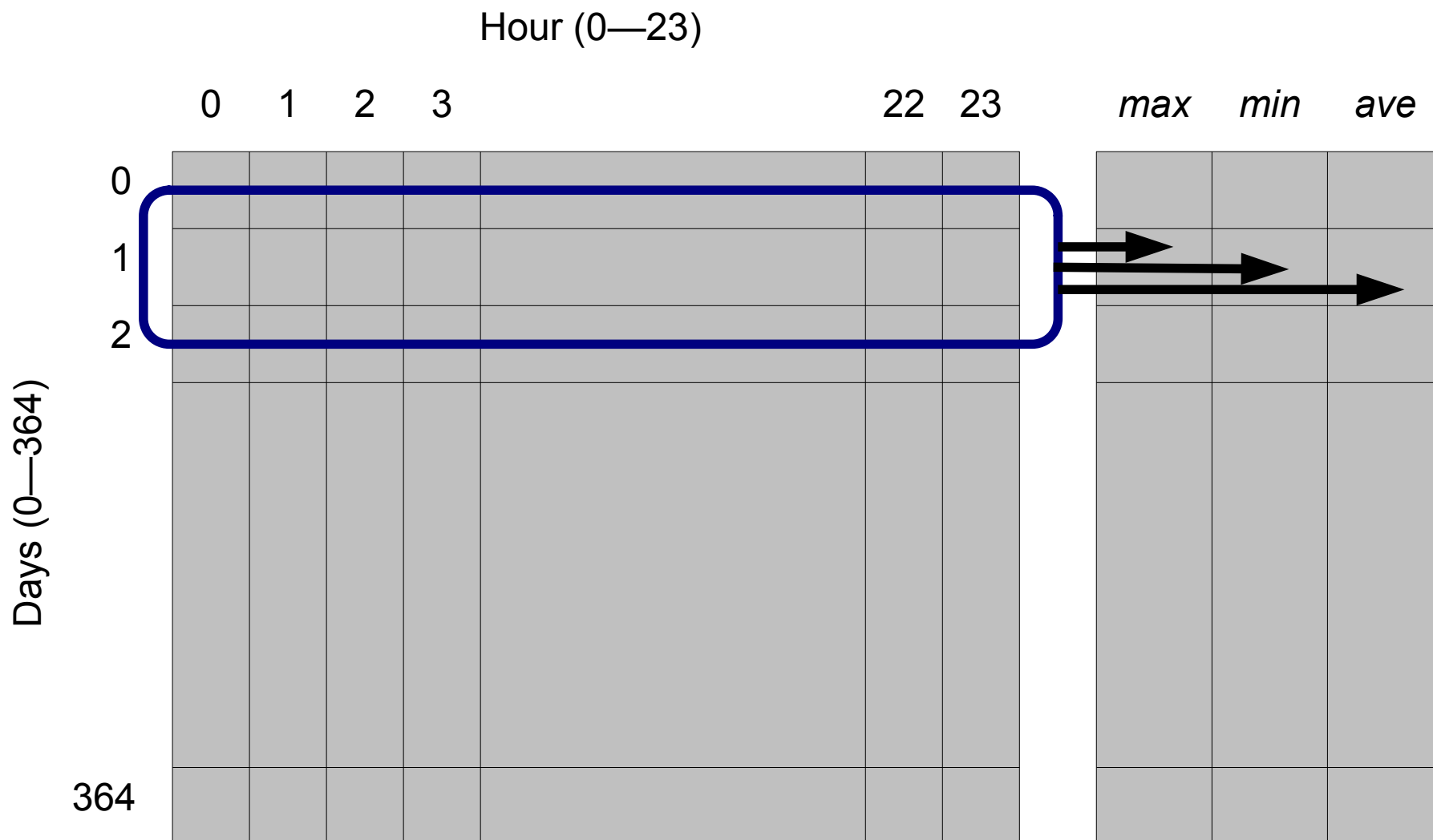
Task parallelism vs Data parallelism

- Task Parallelism
 - Distribute (possibly different) tasks to processors
- Data Parallelism
 - Distribute data to processors
 - Each processor executes the same task on different data

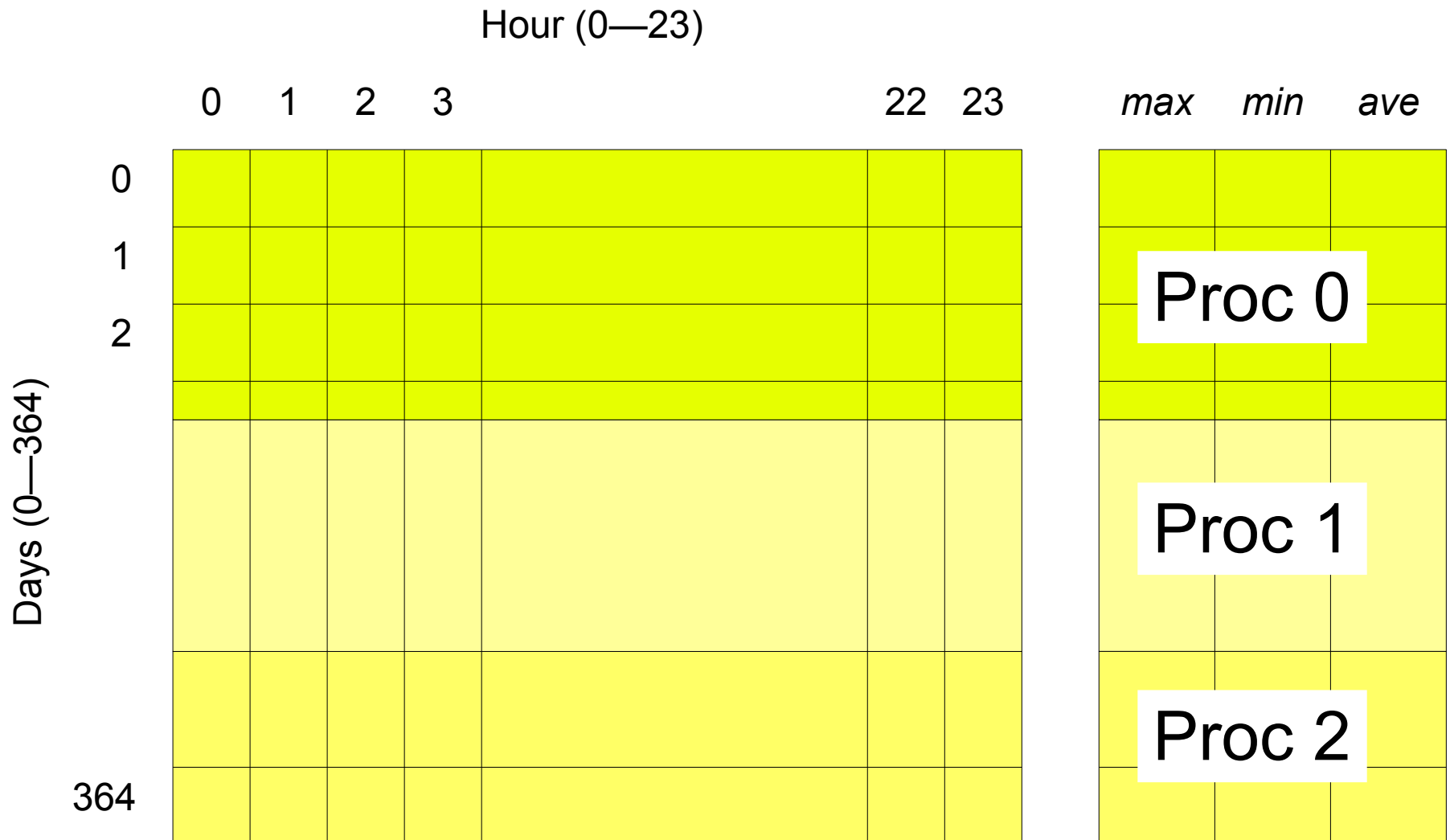
Example

- We have a table containing hourly temperatures on some location
 - 24 columns, 365 rows
- Compute the minimum, maximum and average temperature for each day
- Assume we have 3 independent processors

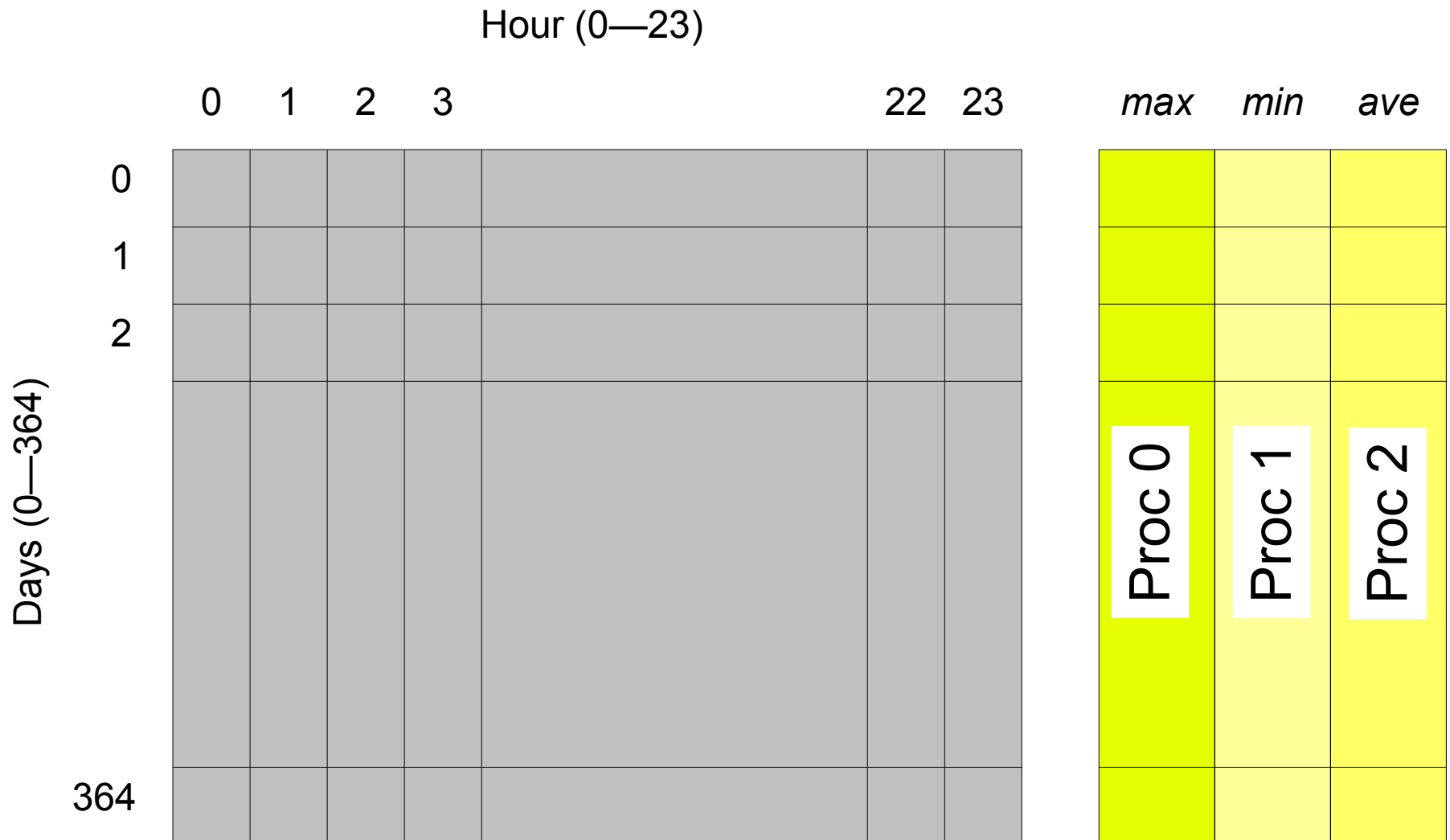
Example



Data parallel approach



Task parallel approach



Introduction to Data Parallel Supercomputing

Thinking Machines Corporation (1989) , Introduction to Data Parallel Supercomputing,

https://www.youtube.com/watch?v=f8NTHfMw_WA

Key concepts

- Parallel architectures “naturally” derive from physics laws
- Parallel architectures require parallel programming paradigms
- Writing parallel programs is much harder than writing sequential programs